



SICAP

CARPETA TECNICA

Proyecto S.I.C.A.P – Carpeta Técnica Oficial

Sistema Integral de Control Automatizado del Pañol (SICAP)

“Orden inteligente, control eficiente”

1. Introducción y Objetivos

El **Proyecto S.I.C.A.P.** (Sistema Integral de Control Automatizado del Pañol) surge como respuesta a un problema cotidiano en talleres, escuelas técnicas y empresas: la **pérdida de herramientas y la falta de trazabilidad** en su uso. Este sistema propone modernizar la gestión del pañol de herramientas mediante tecnología RFID e IoT, evitando pérdidas, mejorando la organización y permitiendo decisiones informadas basadas en datos actualizados .

Objetivo general: Desarrollar e implementar un sistema integral y automatizado de control de activos (herramientas) en un pañol, capaz de **registrar en tiempo real** la entrada y salida de herramientas y quién las toma, eliminando registros manuales y mejorando la eficiencia en la gestión.

Objetivos específicos:

- **Identificación automática por RFID:** Equipar cada herramienta con una etiqueta RFID UHF y cada usuario con una tarjeta/pulsera RFID para detectar su identidad al retirar o devolver herramientas.
- **Detección y registro en tiempo real:** Implementar un portal RFID en la entrada/salida del pañol que, mediante antenas y un lector controlado por ESP32, detecte automáticamente qué herramienta sale o ingresa y qué usuario la porta.
- **Servidor local de datos:** Desplegar un servidor local (Raspberry Pi) con una base de datos integrada que reciba los datos de cada evento (herramienta retirada/devuelta, usuario, fecha y hora) y los almacene automáticamente.
- **Interfaz de gestión web:** Desarrollar una aplicación web accesible desde cualquier dispositivo en la red interna, para visualizar el inventario en tiempo real (qué herramientas están disponibles o en uso, quién las tiene, cuándo fueron retiradas), así



como administrar usuarios y herramientas.

- **Seguridad y control:** Restringir el sistema a una **red local dedicada** para asegurar que solo el personal autorizado pueda acceder, y mantener un registro histórico de movimientos para auditorías o análisis futuros.
- **Integración pedagógica:** Permitir a los desarrolladores (estudiantes) aplicar conocimientos de electrónica, redes y programación en un proyecto integrador real, aprendiendo sobre microcontroladores, bases de datos, backend web y desarrollo frontend, al tiempo que resuelven un problema concreto de su entorno educativo.

2. Justificación Técnica y Pedagógica

Justificación técnica: Muchas instituciones educativas y talleres enfrentan pérdida de herramientas, desorden y falta de control en sus paños. Un sistema automatizado como SICAP mejora significativamente la eficiencia al **eliminar registros manuales y reducir extravíos**, brindando control y responsabilidad compartida. La tecnología **RFID UHF** fue elegida por su capacidad de identificar múltiples objetos a distancia, incluso varios metros, sin requerir línea de vista. Esto la hace ideal para monitorear herramientas en tiempo real sin intervención humana, evitando errores y registrando todo automáticamente. Además, el uso de una arquitectura IoT local (ESP32 + servidor Raspberry Pi) garantiza que el sistema funcione aunque no haya Internet, manteniendo los datos dentro de la institución para mayor seguridad y velocidad de acceso.

Justificación pedagógica: El proyecto SICAP se desarrolló en un contexto escolar técnico, por lo que su implementación tuvo también fines educativos. Los estudiantes involucrados reforzaron competencias en diversas áreas de la tecnología:

- *Electrónica y hardware:* diseño de circuitos (por ejemplo, un conversor RS232-TTL para el lector RFID), integración de sensores/antenas RFID con microcontroladores, y ensamblaje de un portal de detección.
- *Programación embebida:* desarrollo de código en MicroPython para el ESP32, manejando comunicación serial (UART) y protocolos de red (WiFi).
- *Administración de sistemas:* configuración de un servidor Linux (Raspberry Pi), instalación de servicios (servidor web Django, base de datos SQLite) y configuración de redes LAN/WiFi.
- *Desarrollo web:* implementación de un backend con Django (incluyendo APIs REST) y una interfaz web frontend (web app) usando frameworks modernos (Ionic/Angular), aplicando principios de diseño web responsivo.



- *Gestión de proyectos:* documentación técnica completa, pruebas de funcionamiento y trabajo colaborativo, simulando un entorno de desarrollo profesional. Se integraron áreas de software y hardware, lo que brindó a los alumnos una experiencia holística de ingeniería de sistemas.

Desde el punto de vista pedagógico, SICAP sirvió como proyecto integrador que permitió a los estudiantes aplicar en la práctica contenidos de años anteriores y resolver problemas reales de su institución. Asimismo, se fomentó el trabajo en equipo, la resolución de problemas, la creatividad en el diseño de soluciones y la comunicación efectiva (por ejemplo, presentando el proyecto a la comunidad y buscando sponsors para componentes). En resumen, SICAP no solo aporta una **solución tecnológica versátil y funcional** al control de activos físicos, sino que también representa un valioso logro pedagógico al consolidar el aprendizaje multidisciplinar en un proyecto tangible.

3. Alcance y Limitaciones

Alcance del sistema: El alcance de SICAP es **modular y escalable**, lo que significa que puede adaptarse a distintos entornos y escalas de uso. Inicialmente, el proyecto se centró en un *pañol de herramientas* de un taller escolar, pero la misma solución es aplicable a otros escenarios:

- En bibliotecas, para control automatizado de préstamos y devoluciones de libros mediante etiquetas RFID en libros y carnés.
- En depósitos de insumos o almacenes, para monitorear stock en tiempo real y registrar movimientos de inventario automáticamente.
- En laboratorios, para seguimiento de equipos y materiales sensibles.
- En general, cualquier espacio que necesite gestión eficiente y trazable de objetos físicos podría beneficiarse de SICAP.

El sistema abarca todos los componentes necesarios para su funcionamiento autónomo dentro de la institución:

- Un **portal RFID** físico instalado en el acceso al pañol.
- Múltiples **etiquetas RFID UHF** adheridas a cada herramienta, y tags RFID personales para identificar usuarios (p. ej., en tarjetas o pulseras).



- Un **dispositivo lector (ESP32)** conectado a las antenas RFID del portal, encargado de leer los tags y transmitir los datos.
- Una **infraestructura de red local** (router WiFi dedicado) que conecta inalámbricamente el ESP32 con el servidor central.
- Un **servidor central (Raspberry Pi)** ejecutando el backend (Django) y alojando la base de datos local donde se almacenan los registros.
- Una **aplicación web** accesible a través de la red local para consultar el estado del sistema e interactuar (gestionar usuarios, etc.).

Limitaciones del proyecto: Si bien SICAP se diseñó para ser versátil, en su implementación actual presenta las siguientes limitaciones:

- **Alcance físico limitado:** El sistema está pensado para operar en la red local de la institución. No cuenta con acceso remoto vía Internet (aunque podría incorporarse en el futuro), lo que significa que para usar la web app o consultar datos, el dispositivo cliente debe estar conectado al router interno de SICAP.
- **Cobertura RFID específica:** Se emplearon etiquetas y antenas UHF optimizadas para distancias de varios metros. Sin embargo, la efectividad de detección puede variar según el ambiente: presencia de metal u obstáculos, orientación de las etiquetas, interferencias, etc. Es posible que algunas lecturas fallen si una etiqueta está bloqueada o demasiado alejada de la antena. Se calibró el portal para cubrir la entrada del pañol, pero áreas más amplias o múltiples accesos requerirían antenas adicionales (no implementadas en este prototipo).
- **Capacidad de procesamiento local:** La Raspberry Pi y la base de datos SQLite manejan eficientemente la escala de un pañol escolar (decenas de herramientas y usuarios, eventos diarios). En instalaciones mucho más grandes (ej. cientos de herramientas pasando continuamente), podría requerirse un servidor más robusto o una base de datos diferente (PostgreSQL/MySQL) para mejor rendimiento.
- **Seguridad básica:** La red WiFi dedicada emplea contraseña WPA2 para acceso. El backend Django actualmente no implementa autenticación sofisticada para la API de recepción de tags (se permitió envío sin token ni TLS dado el entorno cerrado). Para producción se recomendaría añadir autenticación en la API, cifrado en tránsito (HTTPS) y gestión de roles de usuario en la aplicación web.
- **Disponibilidad de energía y mantenimiento:** El portal RFID requiere que el lector ESP32 y las antenas estén alimentados continuamente mientras el pañol está en uso. Se debe prever una fuente de alimentación estable cerca de la puerta. Asimismo, las etiquetas RFID en herramientas pueden dañarse o desprenderse con el uso rudo; se debe



implementar un mantenimiento periódico para reponer etiquetas defectuosas y calibrar el lector.

- **Integración con sistemas existentes:** Actualmente SICAP funciona de manera autónoma y no está integrado con otros sistemas de la escuela (por ejemplo, no se enlaza con sistemas de gestión escolar). Su adopción masiva requeriría evaluar cómo vincular o exportar datos a plataformas de gestión si fuese necesario.
- **Estado de prototipo:** Este proyecto es un prototipo funcional de fin de curso. Si bien cumple con las funcionalidades básicas esperadas, no ha sido sometido a uso prolongado en un entorno de producción. Algunas características avanzadas quedaron fuera del alcance (ver sección de mejoras futuras), y podrían implementarse en desarrollos posteriores.

4. Arquitectura General del Sistema

Descripción general: La arquitectura de SICAP sigue un modelo IoT local, con componentes distribuidos que se comunican a través de una red interna. A continuación se presenta el diagrama de bloques e interconexiones entre los módulos principales:

- **Etiquetas RFID (UHF):** Cada herramienta del pañol lleva adherida una *etiqueta RFID pasiva UHF on-metal*, con un código único. Asimismo, cada usuario autorizado posee una *pulsera RFID UHF* personal. Estas etiquetas son los identificadores que permiten al sistema distinguir qué objeto y qué persona están interactuando.
- **Portal RFID de acceso:** En la entrada/salida del pañol se instaló un portal con **antenas RFID UHF** capaces de leer las etiquetas a distancia. El portal está compuesto por un marco (similar al de detectores de metales en accesos) donde se montan las antenas de forma que cubren el ancho del paso (ver diseño en la imagen a continuación). Cuando un usuario ingresa o sale por el portal llevando herramientas, las antenas detectan simultáneamente **la etiqueta del usuario y las de las herramientas** en cuestión .

Figura 1: Diseño del portal RFID UHF para el ingreso al pañol. Se muestran las dimensiones estructurales y ubicación de antenas. Este marco permite detectar automáticamente qué herramientas pasan y quién las lleva al atravesar el portal .

- **Módulo ESP32 (Lector RFID):** Un microcontrolador **ESP32** ubicado en el portal actúa como cerebro embebido del lector RFID. Está conectado a las antenas mediante un **módulo lector RFID UHF**. En este proyecto se utilizó un lector cuyo módulo de radiofrecuencia comunicaba por interfaz serial (RS-232) los códigos de etiquetas detectadas. Dado que el ESP32 funciona a niveles TTL, se incorporó un **convertor RS232-TTL** personalizado para interconectar el lector con el ESP32 (ver Figura 2). El



ESP32 se encarga de:

- Leer continuamente los datos enviados por el módulo RFID (códigos de tags de 8 bytes en nuestro caso, recibidos por UART).
 - Filtrar y procesar los eventos de lectura: por ejemplo, evita registrar duplicados inmediatos del mismo tag (misma herramienta pasando varias veces en corto tiempo) usando temporizadores.
 - Determinar cuándo se ha detectado un usuario y una o más herramientas simultáneamente al pasar por el portal.
 - Conectarse a la red WiFi interna y **enviar los datos al servidor**. Cuando se identifica un evento válido (p. ej., *herramienta X retirada por usuario Y*), el ESP32 formatea esa información en un mensaje JSON y lo envía vía HTTP POST al endpoint del servidor Django (ver sección 5.4).
- **Router y Red Local (LAN):** Para la comunicación, se instaló un **router WiFi dedicado** con SSID propio (en nuestro caso llamado "SICAP") y contraseña WPA2. Tanto el ESP32 como la Raspberry Pi servidor se conectan a este router, formando una red local aislada. No se requiere acceso a Internet para el funcionamiento, aunque el router podría estar conectado a Internet para actualizaciones si se desea. La elección de un router dedicado garantiza un canal de comunicación confiable y privado.
 - **Configuración de red:** Se configuró el router con el rango de IP 192.168.111.0/24 (por ejemplo) para evitar conflictos con otras redes comunes. La Raspberry Pi tiene una IP fija (o reservada por DHCP) – en el prototipo se usó 192.168.111.218 – de modo que el ESP32 conoce de antemano a qué dirección enviar los datos del tag leído. Es importante asignar IP fija al servidor para que los dispositivos embebidos puedan apuntar siempre al mismo host.
 - La red interna permite también que otros dispositivos (p. ej. una PC, tablet o smartphone del encargado) se conecten vía WiFi para acceder a la aplicación web de SICAP. Al ser una intranet local, solamente quienes tengan la contraseña WiFi podrán conectarse, añadiendo una capa de seguridad.
 - **Servidor Principal (Raspberry Pi):** El corazón del sistema es una **Raspberry Pi 4** (u otro modelo equivalente), que actúa como servidor **web** y **base de datos** local. La Raspberry Pi se conectó por WiFi al router SICAP (y puede también estar conectada por cable Ethernet al mismo router para mayor estabilidad). En este servidor corren los siguientes servicios:
 - **Backend Django:** Una aplicación web desarrollada con el framework Django (Python) que expone endpoints para recibir datos y una interfaz administrativa



para gestionar la información. El rol del backend es **recibir los mensajes JSON** enviados por el ESP32 con cada lectura de RFID y almacenarlos en la base de datos. Además, provee una API para que la aplicación web de gestión pueda consultar los registros de herramientas, estado de usuarios, etc. (Ver detalles en sección 5.4).

- **Base de datos local:** Se utiliza una base de datos SQL alojada en la misma Raspberry Pi para persistir los datos. Cada evento de retirada/devolución de una herramienta se guarda como un nuevo registro (con ID de tag). Asimismo, se almacenan tablas de usuarios autorizados, herramientas, y otros datos de configuración del sistema. La elección de SQLite simplifica la instalación (no requiere un servidor de BD separado) y es suficiente para el alcance del proyecto. No obstante, la arquitectura Django permite migrar fácilmente a otro motor de base de datos en el futuro si es necesario.
- **Servidor web y aplicación de gestión:** La Raspberry Pi también sirve la **aplicación web de SICAP**, accesible a través de un navegador por URL local (<http://192.168.111.218:5000>). Esta interfaz web permite a un administrador o encargado del pañol visualizar información y administrar el sistema (ver sección 5.5). Django provee la entrega de esta interfaz, ya sea mediante templates renderizados en servidor o sirviendo una aplicación frontend separada. En el prototipo, se implementaron plantillas HTML básicas para login y dashboard, y también se integró un proyecto front-end separado para explorar funcionalidades más avanzadas (descrito más adelante).
- **Dispositivos cliente (Web App):** Cualquier dispositivo (PC de escritorio, laptop, tablet, smartphone) conectado al WiFi interno de SICAP puede acceder a la aplicación web a través del navegador. Esto permite, por ejemplo, que:
 - El bibliotecario o encargado del pañol consulte en su computadora qué herramientas están fuera y con quién.
 - Los docentes desde sus teléfonos vean si hay herramientas disponibles o quién las tomó.
 - Se realicen altas/bajas de herramientas o usuarios desde una interfaz amigable.

El siguiente esquema resume el flujo principal del sistema:

1. **Identificación RFID:** Cuando un usuario atraviesa el portal con herramientas, las antenas RFID leen simultáneamente la etiqueta personal del usuario y las etiquetas de cada herramienta que lleva .



2. **Procesamiento embebido:** El ESP32 recibe los códigos de tags vía UART, los interpreta y determina el evento (ej. *“Herramienta A y B retiradas por Usuario X”*). Aplica un filtro de repetición para evitar múltiples envíos del mismo evento. En nuestro código, si un tag leído es distinto al último enviado o ha transcurrido más de 2 segundos, lo consideramos un nuevo evento válido.
3. **Envío al servidor:** El ESP32 se asegura de estar conectado al WiFi (SSID “SICAP”). Luego envía un mensaje JSON al servidor Django haciendo una petición HTTP POST al endpoint correspondiente (por ejemplo /api/v1/register/tag/) con el contenido {"tag": "<ID_lectura>"}. Cada lectura se envía inmediatamente (con un pequeño retardo programado de ~1 segundo entre lecturas para no saturar).
4. **Registro en base de datos:** Al llegar la petición al backend, la vista de Django correspondiente parsea el JSON, extrae el tag y crea un nuevo registro en la base de datos con el código de tag y la fecha/hora actual. De esta forma queda persistente la operación. Si el tag corresponde a un usuario o herramienta conocidos, se puede enlazar esa referencia (en este prototipo, por simplicidad, se almacenan los tags crudos; un desarrollo futuro podría tener tablas separadas y relaciones, ver sección 6).
5. **Visualización web:** Los registros guardados pueden ser consultados desde la aplicación web. Por ejemplo, un módulo de *“Movimientos”* listaría las últimas entradas/salidas con campos de herramienta, usuario, hora, etc. También se ofrece al usuario de la web app una vista general del **inventario actualizado en tiempo real**: qué herramientas están en el pañol y cuáles están actualmente prestadas (y en manos de quién) . Esto es posible porque cada vez que llega un registro nuevo, la base de datos refleja el cambio de estado de esa herramienta (pasó de *disponible* a *en uso*, o viceversa). La web app puede consultar periódicamente o vía solicitud del usuario para refrescar estos datos.

Esta arquitectura modular garantiza que cada subsistema (embebido, red, servidor, frontend) pueda desarrollarse y diagnosticarse de forma independiente, a la vez que se integran en un todo funcional.

5. Descripción Detallada de los Módulos

A continuación, se describen en detalle todos los módulos y componentes del proyecto SICAP, incluyendo hardware y software, con ejemplos de código, esquemas y consideraciones técnicas para replicar cada parte.

5.1 Módulo ESP32 (Lector RFID)

Rol en el sistema: El módulo basado en ESP32 funciona como **dispositivo lector RFID inteligente** en el portal. Su responsabilidad es captar los datos crudos de las etiquetas RFID a



través del lector/antena y procesarlos mínimamente para luego enviarlos al servidor central. Esto descarga al servidor de la tarea de lidiar con la comunicación de bajo nivel con el hardware RFID.

Hardware utilizado:

- **Microcontrolador ESP32** (modelo ESP32-WROOM-32 dev kit): seleccionado por su conectividad WiFi integrada, su buen desempeño y su familiaridad en programación (soporta entornos Arduino C++ o MicroPython). Se alimenta con 5V (por el cable USB o fuente externa) y regula internamente a 3.3V.
- **Módulo lector RFID UHF externo:** se empleó un lector UHF de largo alcance que comunica las lecturas vía interfaz serial RS-232. Este módulo está conectado a antenas UHF (al menos 2 antenas para cubrir ambos lados del portal). Los detalles específicos del modelo pueden variar; en nuestro prototipo se utilizó un kit con antenas tipo panel UHF y un módulo lector capaz de leer tags EPC Class1 Gen2.
- **Conversor RS-232 a TTL:** dado que la UART del ESP32 maneja niveles TTL (3.3V) y el lector entregaba RS-232 (+/- 12V), se fabricó un pequeño conversor basado en MAX3232. Este circuito incluye un MAX3232CPE y capacitores de bomba de carga para traducir las señales RX/TX. El conversor se implementó en una placa impresa casera (Figura 2) con un conector DB9 hacia el módulo lector y pines hembra hacia el ESP32 (GPIO17 y GPIO16 utilizados para RX/TX respectivamente en el UART2 del ESP32).

Figura 2: Fotografía del conversor RS232-TTL diseñado para conectar el lector RFID serial al ESP32. Se observa el chip MAX3232 y el conector DB9 para la interfaz RS-232 del módulo RFID. Este conversor adapta los niveles de voltaje y permite al ESP32 comunicarse con el lector sin dañar sus puertos.

- **Antenas RFID:** antenas UHF polarización circular, montadas en el marco del portal. Conectadas al módulo lector mediante coaxiales (tipo SMA/N). Se ubicaron de forma enfrente la zona de paso, garantizando un campo de lectura adecuado.
- **Fuente de alimentación:** El ESP32 y el conversor se alimentan con 5V (tomado del USB del ESP32 o una fuente regulada). El módulo lector RFID UHF suele requerir 12V DC y consume corriente significativa al activar las antenas (varios amperios pico); se usó una fuente de 15V 4A dedicada para el lector. Importante conectar todas las tierras comunes.

Conexiones principales:

- ESP32 GPIO16 (RX2) $\leftrightarrow$ TX del conversor (desde lector RFID).



- ESP32 GPIO17 (TX2) $\Leftrightarrow$ RX del conversor (hacia lector RFID) – aunque en nuestro caso el lector se configura para *output* continuo de tags, por lo que la comunicación es principalmente unidireccional. Aun así, se dejó conectada la vía de transmisión por si se requiere enviar comandos al lector (p.ej. para configurar potencia, inicio/parada de lectura).
- ESP32 3.3V $\Leftrightarrow$ Vcc del MAX3232 (el MAX3232 opera a 3.3V).
- Conversor DB9 $\Leftrightarrow$ Puerto serie del lector RFID.
- ESP32 VIN (5V) $\Leftrightarrow$ Fuente 5V (alimentación del ESP32).
- Lector RFID 12V $\Leftrightarrow$ Fuente 12V.
- Todos GND unidos (ESP32 GND, conversor GND, lector GND, fuente negativa común).

Programación del ESP32:

Se optó por programar el ESP32 usando **MicroPython** por su rapidez de desarrollo y facilidad de legibilidad, aunque una alternativa podría haber sido usar C/C++ (framework Arduino ESP32). MicroPython ofreció la ventaja de contar con librerías como urequests para realizar fácilmente peticiones HTTP.

El código se estructuró en tres archivos principales cargados en el ESP32:

- wifi.py – encargado de conectar el ESP32 a la red WiFi **SICAP** al iniciar.
- Reader.py – clase RFIDReader para manejar la UART y leer bytes de tags.
- app.py – bucle principal que combina todo: inicializa WiFi, instancia el lector, espera detecciones y envía peticiones al servidor.

Conexión WiFi: En wifi.py definimos la función de conectar, fijando SSID y password de la red. Por simplicidad, estos están codificados como constantes en el código:

Fragmento de wifi.py

```
import network
import time

def connect_to(ssid : str, passwd : str) -> None:
    """Conecta el microcontrolador a la red indicada.
```



```
Parameters
-----
ssid : str
    Nombre de la red a conectarse
passwd : str
    Contraseña de la red
"""

import network
from time import sleep

sta_if = network.WLAN(network.STA_IF)
if not sta_if.isconnected():
    print("Connecting to network...")
    sta_if.active(True)
    sta_if.connect(ssid, passwd)
    while not sta_if.isconnected():
        print(".",end="")
        sleep(.05)

print()
print("Network config:", sta_if.ifconfig())
print()
```

Cuando se energiza el ESP32, intenta unirse a la red WiFi interna. Si no lo logra en el lapso dado, se imprime un error crítico y (en nuestro código) se detiene el programa, dado que sin conexión no tendría sentido leer etiquetas. En la práctica, se podría programar reintentos indefinidos.

Lectura de tags por UART: La clase RFIDReader configura la UART2 del ESP32 a 57600 baudios (protocolo del lector RFID). Por ejemplo:

```
from machine import UART
import time

class RFIDReader:
    def __init__(self, tx=17, rx=16, baudrate=57600, uart_id=1):
        # Ajusta uart_id/pines/baud según tu hardware
        self.uart = UART(uart_id, baudrate=baudrate, tx=tx, rx=rx,
                        timeout=100, timeout_char=20)
        self._buf = "" # acumulador de texto
```



```
def _extract_epc_from_text(self, text):

    start = text.find("d:")
    if start == -1:
        return None
    start += 2
    # Buscar fin de campo
    end_candidates = []
    for ch in (',', ';', '}'):
        pos = text.find(ch, start)
        if pos != -1:
            end_candidates.append(pos)
    if not end_candidates:
        return None
    end = min(end_candidates)
    epc = text[start:end].strip().upper()
    # Validamos que sea HEX (0-9 A-F)
    if epc and all(c in "0123456789ABCDEF" for c in epc):
        return epc
    return None

def read_tag(self):
    """
    Lee del UART, acumula y devuelve un EPC (Tag ID) si aparece.
    Si no hay EPC completo, devuelve None.
    """
    data = self.uart.read()
    if data:
        try:
            chunk = data.decode('latin1', 'ignore')
        except:
            chunk = ""
        if chunk:
            self._buf += chunk

        # Procesar por líneas si vienen \r\n
        if '\n' in self._buf or '\r' in self._buf:
            lines = self._buf.splitlines(True) # conserva
separadores

            # Dejo el último fragmento (posible incompleto) en el
buffer

            tail = ""
```



```

        for ln in lines:
            if ln.endswith('\n') or ln.endswith('\r'):
                epc = self._extract_epc_from_text(ln)
                if epc:
                    return epc
            else:
                tail += ln
        self._buf = tail

# Por si vienen sin saltos de línea, buscar en el buffer
entero

epc = self._extract_epc_from_text(self._buf)
if epc:
    # recorto el buffer para que no crezca infinito
    self._buf = self._buf[-256:]
    return epc

# limpieza preventiva
if len(self._buf) > 4096:
    self._buf = self._buf[-512:]

return None

```

En nuestro caso, cada tag RFID UHF proporciona un identificador único de 8 bytes que representamos en formato hexadecimal. La función `read_tag()` devuelve el código del tag leído (por ejemplo "4B01020304AABBCC") o `None` si no se leyó nada en ese instante. El módulo RFID está en modo *continuous read*, enviando el ID inmediatamente al detectar una etiqueta nueva dentro del rango.

Lógica principal de envío de datos: En `app.py` se combinan la conexión WiFi, la lectura de tags y el envío HTTP. Un pseudocódigo simplificado:

1. Conectar WiFi (usando `wifi.conectar()`).
2. Imprimir "Listo. Esperando tags..." e iniciar bucle infinito.
3. En cada iteración del loop, llamar `tag_detectado = lector.read_tag()`.
4. Si `tag_detectado` no es `None`, entonces:
 - Obtener timestamp actual (`time.time()`).



- Si el tag es distinto al último enviado o ha pasado más de TIMEOUT_TAG segundos desde el último envío, procesarlo:
 - Imprimir por consola del ESP32 el tag detectado.
 - Formar un JSON {"tag": tag_detectado}.
 - Hacer response = urequests.post(API_URL, json=json_data) donde API_URL es la URL del endpoint del servidor (por ejemplo http://192.168.111.218:8000/api/v1/register/tag/ en nuestro caso).
 - Verificar response.status_code: si está en rango 200 (OK), imprimir confirmación; si es error (>=400), imprimir código y texto de error.
 - Llamar response.close() para liberar la conexión HTTP.
 - Guardar ultimo_tag_enviado = tag_detectado y tiempo_ultimo_envio = tiempo_actual para el control de duplicados.
 - Si el tag es igual al último y ocurrió muy recientemente, se ignora (esto evita inundar de lecturas repetidas cuando una misma etiqueta permanece en rango).
5. Volver al inicio del loop (pequeño delay interno de 1 seg entre iteraciones para no saturar CPU).

```
import time
from Reader import RFIDReader
import urequests
import wifi

SSID = "SICAP"
PASSWORD = "graciasamigo"
SERVER_IP = "192.168.111.218" # Django
API_URL = f"http://{SERVER_IP}:5000/api/v1/register/tag/"

# Parámetros de robustez
LOOP_DELAY_MS = 50          # 20 Hz de chequeo
STABLE_READS = 3            # lecturas consecutivas iguales para considerar "estable"
ABSENCE_MS = 600            # tiempo sin leer nada = tag retirado

lector = RFIDReader(tx=17, rx=16)
```



```
# Estado
current_tag = None          # tag "latched" (enviado y sostenido)
stable_tag = None          # último tag visto para conteo de estabilidad
stable_count = 0           # lecturas consecutivas del mismo tag
last_seen_ms = time.ticks_ms() # última vez que se leyó ALGÚN tag

print("Iniciando programa...")
wifi.connect_to(SSID, PASSWORD)
print("\nListo. Esperando tags...")

def send_tag(tag_val):
    try:
        payload = {"tag": tag_val}
        resp = urequests.post(API_URL, json=payload)
        status = resp.status_code
        body = ""
        try:
            body = resp.text
        except Exception:
            pass
        resp.close()

        if 200 <= status < 300:
            print("✅ Servidor respondió: Tag registrado.")
        else:
            print(f"❌ Error del servidor: {status} {body}")

    except Exception as e:
        print(f"❌ Error de conexión al enviar el tag:", e)

while True:
    now = time.ticks_ms()
    tag_detectado = lector.read_tag() # Devuelve el EPC (Tag ID) o None

    if tag_detectado:
        print(tag_detectado) # 📡 imprime el Tag ID crudo

        # Actualizamos la última vez que vimos un tag
        last_seen_ms = now

        # Debounce por lecturas consecutivas
        if tag_detectado == stable_tag:
```



```

        stable_count += 1
    else:
        stable_tag = tag_detectado
        stable_count = 1

    # Si el tag se mantiene estable, lo enviamos al servidor
    if current_tag is None and stable_count >= STABLE_READS:
        print(f"Tag detectado estable: {stable_tag}. Enviando al
servidor...")
        send_tag(stable_tag)
        current_tag = stable_tag # quedamos en HELD

    else:
        # Si no hay tag leído y antes había uno, chequeamos si se retiró
        if current_tag is not None and time.time_diff(now, last_seen_ms) >
ABSENCE_MS:
            print("Tag retirado. Listo para próximo registro.")
            current_tag = None
            stable_tag = None
            stable_count = 0

time.sleep_ms(LOOP_DELAY_MS)

```

(Notar que en el código del prototipo se usó un endpoint dentro de /admin/ por simplicidad, aunque finalmente se habilitó uno en /api/v1/. Lo correcto es usar el endpoint de API para no depender de la interfaz admin de Django).

Consideraciones de programación del ESP32:

- El ESP32 se programó cargando MicroPython firmware primero. Luego mediante la herramienta **Thonny** IDE o ampy, se subieron los archivos .py al sistema de archivos interno del ESP32.
- Se configuró un main.py para que al reiniciar el ESP32 arranque automáticamente el script principal (app.py). Así, en cuanto recibe alimentación, el dispositivo se conecta al WiFi y empieza a monitorear y reportar tags sin intervención manual.
- Se implementaron *prints* de depuración en serie (como se ve en el código) de modo que durante el desarrollo, conectando el ESP32 por USB al PC, se podía ver en la consola (REPL) qué tags leía y las respuestas del servidor.



- La gestión de errores incluyó reconectar WiFi en caso de desconexión y manejo de excepciones en el envío HTTP (por ejemplo, si el servidor no responde, se imprime el error pero el ESP32 sigue funcionando y tratando de reenviar en la siguiente detección).
- **Timing:** Dado que el lector UHF puede detectar muchas etiquetas casi simultáneamente, un posible reto es leer múltiples tags juntos. En esta versión, el código `read_tag()` está orientado a leer de a uno (asumiendo que el módulo RFID envía un tag ID a la vez). Si hubieran múltiples, el módulo suele enviar secuencialmente varias tramas. El ESP32 las irá procesando en bucle. Existe la posibilidad de que si pasan 5 herramientas juntas, se envíen 5 POST seguidos. Esto fue considerado aceptable para nuestro caso ya que la red local puede soportarlo. Sin embargo, si se quisiera optimizar, podría enviarse un solo POST con una lista de tags detectados en un intervalo corto, reduciendo carga. Este tipo de optimización quedó como mejora futura no implementada.

En resumen, el módulo ESP32 actúa como puente entre el **mundo físico RFID** y el **backend digital**, dotando al sistema de la capacidad de *sentir* qué ocurre en el pañol (qué se mueve y quién lo hace) y comunicarlo en tiempo real al servidor.

5.2 Módulo Raspberry Pi (Servidor Principal)

Función

La **Raspberry Pi** es el **servidor central** de SICAP: aloja la lógica de backend (Django), la base de datos y expone las **APIs REST** consumidas por la web app y por el ESP32. Permanece encendida durante el horario de uso del pañol, lista para **recibir lecturas RFID** y **atender consultas** desde la interfaz web.

Hardware y SO

- **Raspberry Pi 4 Model B (4 GB RAM).** El sistema es liviano; podría correr en una Pi 3.
- **Raspberry Pi OS (Bullseye, 32 bits)**, basado en Debian.
- Conexión recomendada: **Ethernet** al router (más estable); alternativa: **Wi-Fi**.
- Configuración inicial con monitor/teclado; operación habitual por **SSH**.
- IP reservada en el router para acceso consistente en LAN (ej. **192.168.111.218**).



Instalación del entorno software

- **Python 3 + venv** para aislar dependencias del proyecto.
- **Django** como framework del backend.
- **Django REST Framework (DRF)** para las APIs.
- **corsheaders** para permitir solicitudes desde el front Angular cuando corre en otro origen/puerto.
- **WhiteNoise** para servir **estáticos** sin depender de Nginx.
- **Gunicorn** como servidor WSGI de producción.
- **systemd** para **arranque automático** y **reinicio** del servicio si falla.
- Base de datos por defecto: **SQLite3** (suficiente para el alcance del proyecto; escalable a PostgreSQL/MariaDB si se requiere).

Código del backend

Estructura (carpetas principales del proyecto *SICAP_Backend*):

- `sicap_backend/` (proyecto): `settings.py`, `urls.py`, `wsgi.py`.
- `registros/` (app principal):
 - `models.py`: modelo `RegistroTag`.
 - `views.py`: vistas de **ingesta de tags**, **listado** y **edición**.
 - `serializers.py`: serialización DRF (cuando aplica).
 - `urls.py`: rutas del módulo.
- `usuarios_login/` (autenticación/gestión de usuarios): vistas y serializers básicos para login y administración.



- [templates/](#) (plantillas simples de apoyo para prototipos).

Modelo de datos (extracto conceptual, basado en tu implementación):

```
from django.db import models
from django.utils import timezone
class RegistroTag(models.Model):
    CATEGORIA_AELEGIR = (
        ('persona', 'Persona'),
        ('insumo', 'Insumo'),
    )
    nombre = models.CharField(max_length=100, blank=True, null=True)
    tag = models.CharField(max_length=100)
    fecha_hora = models.DateTimeField(default=timezone.now)
    categoria = models.CharField(max_length=20, choices=CATEGORIA_AELEGIR, blank=True,
    null=True, default=None )
    def __str__(self):
    return f"{self.tag} - {self.fecha_hora}"
class Panol(models.Model):
    nombre = models.CharField(max_length=100)
    icono = models.CharField(max_length=50, default='cube')
    def __str__(self):
    return self.nombre
class Asignacion(models.Model):
    persona_tag = models.CharField(max_length=100)
    persona_nombre = models.CharField(max_length=100, blank=True, null=True)
    item_tag = models.CharField(max_length=100)
    item_nombre = models.CharField(max_length=100, blank=True, null=True)
    asignado_en = models.DateTimeField(default=timezone.now)
    devuelto_en = models.DateTimeField(blank=True, null=True)
    activo = models.BooleanField(default=True)
    def __str__(self):
    estado = "activo" if self.activo else "devuelto"
    return f"{self.persona_tag} <- {self.item_tag} ({estado})"
```

Nota: **nombre** y **categoria** son opcionales para permitir personalización gradual;
tag es el identificador efectivo leído por el ESP32.

Configuración de Django (puntos clave)



- **ALLOWED_HOSTS:** incluir la IP local de la Pi y `localhost` para acceso LAN.
- **Zona horaria:** `TIME_ZONE = "America/Argentina/Buenos_Aires"` y `USE_TZ = True` para almacenar en UTC y **mostrar** en hora local coherente.
- **Archivos estáticos:** `WhiteNoise` en `MIDDLEWARE` y `STATIC_ROOT` definido para poder coleccionar/servir CSS/JS del admin y de la app.
- **CORS:** habilitado (seguro en LAN) cuando el front se sirve desde otro origen (Angular/Ionic).
- **REST Framework:** permisos por defecto conservadores y relajados en los endpoints públicos (p. ej., ingesta de tags) según necesidad del prototipo.

Despliegue y servicio (producción en la Pi)

- **Gunicorn** ejecuta el proyecto Django en `0.0.0.0:5000`, permitiendo acceso desde la LAN.
- **systemd** gestiona un servicio llamado **sicap.service**:
 - Se inicia automáticamente al encender la Pi.
 - **Reinicia** el proceso si se cae.
 - **Depende de la red** (arranca cuando la red está disponible).
- **WhiteNoise** sirve estáticos (admin y front compilado si se integra) sin necesidad de Nginx.
- **Logs:** se consumen con `journalctl` para ver peticiones y errores en tiempo real, útil para diagnóstico con el ESP32 y la web app.

Resultado: el servidor **queda siempre prendido** y listo para recibir lecturas y atender la web sin intervención manual.

Endpoints principales del backend (visión funcional)



- **Ingesta de tags** (</api/v1/register/tag/>):
 - Método: **POST** (desde ESP32).
 - Cuerpo: JSON con "tag".
 - Efecto: crea un **RegistroTag** y responde estado.
- **Listado rápido para la web** (</api/v1/register/tag/list/>):
 - Método: **GET**.
 - Devuelve: **id**, **tag**, **nombre**, **categoría** y **created_at** (hora local formateada).
- **Edición/Personalización** (</api/v1/register/tag/<id>/editar/>):
 - Método: **PUT/PATCH**.
 - Cuerpo: puede incluir "nombre" y/o "categoría".
 - Efecto: actualiza el registro (sin cambiar el **tag** salvo que se envíe explícitamente).
- **Admin de Django** (</admin/>):
 - Gestión visual de **RegistroTag**, usuarios, etc., pensado para desarrollo/soporte.

(Otros endpoints de [usuarios_login/](#) permiten login y altas simples de usuarios en el entorno del prototipo.)

Interacción de módulos

- **ESP32 → Raspberry (Django):**
El microcontrolador envía un **POST** con el **tag** leído; el backend registra el evento con timestamp y opcionalmente lo muestra nombrado si se personalizó previamente.
- **Web App → Raspberry (Django):**
La web consulta el **listado** para renderizar tablas y llama a **editar** para asignar



`nombre/categoria` a cada `tag`.

Consideraciones de operación y mantenimiento

- **Hora correcta:** la Pi usa la zona `America/Argentina/Buenos_Aires` y sincroniza por NTP; la API convierte la hora a **local** al exponer `created_at`.
 - **Resiliencia:** `systemd` mantiene el servicio arriba; los logs permiten diagnóstico rápido.
 - **Seguridad:** en LAN, CORS habilitado para el front; en despliegues más abiertos conviene limitar orígenes y activar autenticación para endpoints críticos.
 - **Escalabilidad:** la base SQLite del prototipo es migrable a PostgreSQL/MariaDB; el esquema y el uso de Django/DRF permiten extender modelos (p. ej., entidad **Herramienta/Usuario** con relaciones) sin romper APIs existentes.
-

Conclusión

La Raspberry Pi se configuró como **servidor de producción liviano**: **Django + DRF** para la lógica y APIs, **Gunicorn** como WSGI, **WhiteNoise** para estáticos y **systemd** para **autoarranque y recuperación**. Esta arquitectura hace que SICAP sea **confiable** (siempre disponible), **mantenible** (logs y admin) y **extensible** (campos `nombre` y `categoria`, nuevos modelos y vistas) dentro de un contexto escolar/industrial ligero.



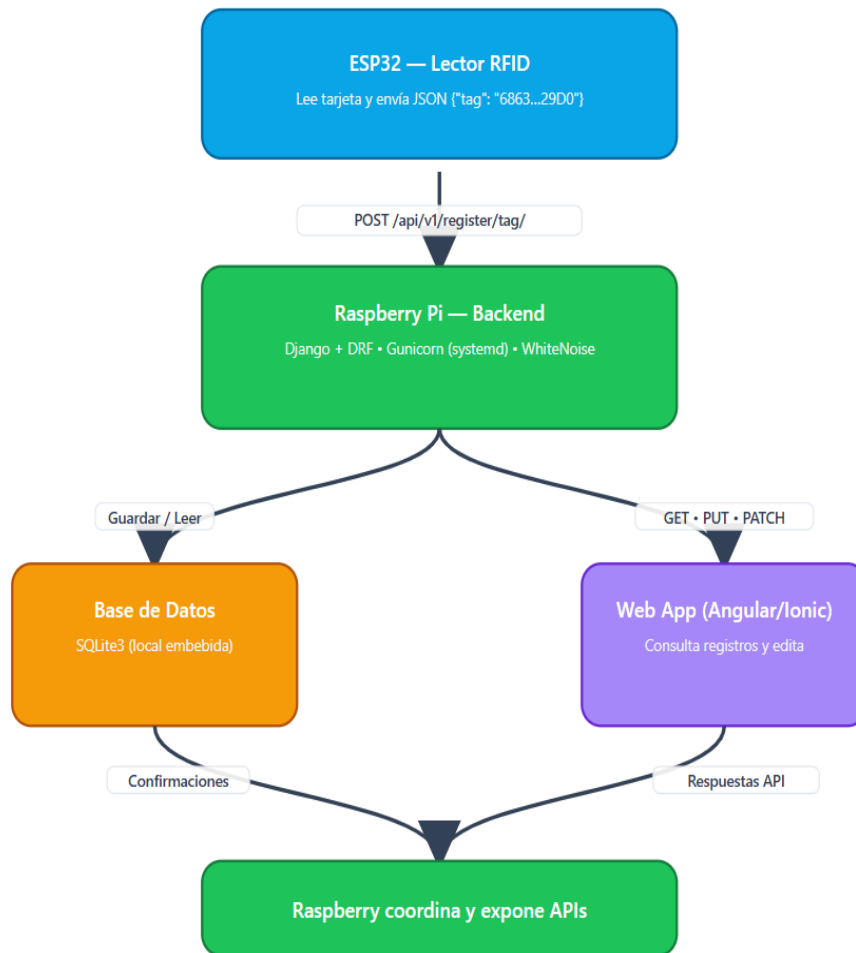


Figura 5.2: Flujo de comunicación en el módulo Raspberry Pi. El ESP32 captura el tag RFID y lo envía mediante una petición POST al servidor Django corriendo en la Raspberry Pi (gestionado por Gunicorn y systemd). El backend procesa y almacena los datos en la base de datos SQLite3 y expone endpoints REST. La Web App (Angular/Ionic) consume estos endpoints para consultar los registros y permitir operaciones como edición o gestión de los tags.

5.3 Módulo Router (Red Interna)

El **router dedicado** para la red interna de SICAP es un componente sencillo pero fundamental para la comunicación entre los demás módulos. Sus funciones principales son:

- Proveer una **red WiFi local segura** (SSID propio y contraseña) para conectar el ESP32 y la Raspberry Pi (y eventualmente dispositivos clientes).



- Asignar direcciones IP locales y enrutar la comunicación dentro de la LAN.
- Aislar el sistema de la red institucional general o de internet, salvo que se requiera lo contrario, para mayor seguridad y estabilidad.

Configuración realizada:

- Se utilizó un router TP-Link TL-WR840N (por ejemplo) operando en modo estándar (NAT activado, servidor DHCP activo). Se configuró el **SSID = “SICAP”** y **clave WPA2-PSK = “graciasamigo”** (como se ve en el código WiFi del ESP32) para que ambos dispositivos pudieran autenticarse inalámbricamente.
- **Rango IP:** Se definió el rango 192.168.111.1 – 192.168.111.254 para la red. El router tomó la IP 192.168.111.1. Esto se hizo para evitar colisión con otras redes comunes (como 192.168.0.x o 192.168.1.x) y distinguir claramente la subred de SICAP.
- **DHCP:** Activo en el router, entregando IPs automáticamente. Sin embargo, se reservó una IP fija para la Raspberry Pi para facilitar su localización. En las opciones DHCP del router, se asoció la dirección MAC de la Raspberry Pi a la IP **192.168.111.218** (en el prototipo). Así, cada vez que la Pi se conecta, recibe esa IP. Esto es importante porque el ESP32 necesita conocer a qué IP enviar los datos. Alternativamente, se pudo configurar la IP estática directamente en la Raspberry Pi, pero la reserva DHCP es conveniente centralmente.
- **Puerto de servidor:** El backend corre en el puerto 8000. Dentro de la LAN, no se requiere abrir puertos manualmente ya que el tráfico es local. Si se quisiera acceder desde fuera (por ejemplo monitorear desde internet), habría que configurar *port forwarding* (redireccionar el puerto 8000 a la IP de la Pi) y asegurarse de seguridad (idealmente VPN o firewall, ya que exponerlo públicamente no es recomendable sin auth fuerte). En este proyecto escolar, se decidió **no exponer externamente** el servidor.
- **Sin conexión a internet:** El router puede estar desconectado de internet (no tiene WAN). Todos los servicios funcionan offline. Esto es deliberado para que la dependencia sea nula de factores externos. Adicionalmente, al no haber salida a internet, se previene que el ESP32 intente resoluciones DNS o actualizaciones. Todo se maneja por IP locales. *(Nota: Durante desarrollo, a veces se conectó temporalmente el router a internet para instalar paquetes en la Pi; pero en producción no haría falta.)*

Cobertura WiFi y posicionamiento: El router se colocó en proximidad del pañol para garantizar buena señal. Tanto la Raspberry Pi como el ESP32 estaban en rango (unos pocos metros). El ESP32 en particular, con antena integrada, tenía señal fuerte a esa distancia. Si el pañol estuviera en un taller grande, se puede considerar ubicar el router en un punto central o



usar extensores, pero usualmente la cobertura de un router hogareño es suficiente para un área de decenas de metros.

Seguridad de la red: Con WPA2 se asegura que solo personal autorizado con la contraseña pueda unirse. No se entregó la clave a cualquiera, solo al equipo del proyecto y docentes responsables, para evitar intrusos en la red. Por otro lado, dentro de la LAN, el protocolo de envío de tags es sin cifrar (HTTP plano). Esto fue considerado aceptable ya que para interceptarlo alguien tendría que haber roto la WiFi primero. Una mejora sería usar HTTPS incluso en LAN (certificado autofirmado en Pi), pero implica complejidad adicional de manejo de certs en MicroPython. Otra medida implementada: el endpoint de recibir tag está oculto tras `/api/v1/` sin anuncio público, y la app web de gestión requiere login. Con esto, aunque alguien se conecte a la WiFi, no puede alterar datos fácilmente.

Configuración de red en Raspberry Pi: La Pi se configuró para usar la IP fija al boot (via DHCP reservado). También se editó el hostname a “sicap-server” para identificarla en la red. El archivo `/etc/hosts` se ajustó para que sicap-server resolviera a 192.168.111.218 localmente. Esto permitió, por ejemplo, que en el navegador de un cliente se pudiera teclear `http://sicap-server:8000` en vez de memorizar la IP (alternativamente se podría usar mDNS/Bonjour con `hostname.local`). Este detalle es opcional pero aporta comodidad.

Pruebas del router y red: Se verificó que:

- El ESP32 obtenía IP (usando `wlan.ifconfig()` se pudo ver su IP, máscara, gateway).
- El ESP32 podía hacer ping al Raspberry Pi y viceversa (MicroPython no tiene ping fácil, pero se deduce por el éxito de las peticiones HTTP).
- Latencia en la LAN < 2 ms, pérdidas 0%. Esto asegura que los POSTS del ESP32 llegan casi instantáneamente.
- En la Raspberry Pi, se pudo resolver nombres si se necesitaba (aunque sin internet, solo es relevante para mDNS).
- No hubo interferencias ya que se usó un canal WiFi poco congestionado (el router en canal 6, y se aisló de otras redes cercanas).

En resumen, el router configura un **entorno de red controlado** para que los dispositivos de SICAP se comuniquen de forma eficiente y segura. Para replicar el proyecto, cualquier router inalámbrico básico sirve, siempre que se configure el SSID/password esperados y se fije la IP del servidor. En caso de modificar esos datos, recordar cambiarlos también en el código del ESP32 y en la documentación.

5.4 Backend en Django (Lógica del Servidor)



El backend de **SICAP** está construido con **Django** y **Django REST Framework (DRF)**. Expone endpoints REST para:

1. recibir lecturas RFID desde el ESP32,
2. listar/editar esas lecturas,
3. administrar **Pañoles** (perfiles/espacios del sistema) consumidos por la web-app (Ionic/Angular).

La API está **versionada** bajo `/api/v1/` y entrega/acepta **JSON**.

5.4.1 Estructura de proyecto (módulo **registros**)

```
sicap_backend/                                # Proyecto Django
├─ sicap_backend/
│   ├─ settings.py                            # Config global (CORS/CSRF/DRF, apps)
│   └─ urls.py                                # Monta /api/v1/ → registros.urls
└─ registros/                                # App de RFID + Pañoles
    ├─ models.py                             # Modelos de datos
    ├─ serializer.py                         # Serializadores DRF
    ├─ views.py                              # Vistas (funcionales) y ViewSets (DRF)
    ├─ urls.py                               # Router + endpoints
    └─ admin.py                             # Registro en Django Admin
```

El **versionado** en `/api/v1/` permite agregar breaking changes en `/api/v2/` sin romper integraciones vigentes.



5.4.2 Modelos de datos

RegistroTag

Representa una **lectura RFID**.

```
# registros/models.py

from django.db import models

from django.utils import timezone

class RegistroTag(models.Model):

    nombre = models.CharField(max_length=100, blank=True, null=True)

    tag = models.CharField(max_length=100, db_index=True)

    fecha_hora = models.DateTimeField(default=timezone.now)

    categoria = models.CharField(max_length=100, blank=True,
null=True)

    def __str__(self):

        return f"{self.tag} - {self.fecha_hora:%Y-%m-%d %H:%M:%S}"
```

- **tag** está **indexado** para consultas rápidas (p. ej. filtrar por tag).
- **nombre/categoria** permiten **anotar** lecturas (ej: T99999 → “Taladro”, U12345 → “Juan Pérez”).
- **fecha_hora** usa **timezone.now** para generar un timestamp en el servidor (evita depender del reloj del dispositivo).

Panor



Espacio/“perfil” administrable en el menú.

```
# registros/models.py
```

```
class Panol(models.Model):

    nombre = models.CharField(max_length=100, unique=True)

    icono = models.CharField(max_length=50, default='cube')

    def __str__(self):

        return self.nombre
```

- **nombre** es **único** (dos paños no deben llamarse igual).
- **icono** acompaña el estilo tipo “Netflix” del front.

Extensibilidad: si más adelante necesitás “Herramienta” o “Operario”, se agregan modelos relacionados (FK) sin tocar los contratos existentes.

5.4.3 Serializer (DRF)

Los **serializers** definen el contrato JSON expuesto/aceptado por la API.

```
# registros/serializer.py

from rest_framework import serializers

from .models import Panol, RegistroTag

class PanolSerializer(serializers.ModelSerializer):

    class Meta:
```



```

model = Pano1

fields = ['id', 'nombre', 'icono']

class RegistroTagSerializer(serializers.ModelSerializer):

    # Alias de fecha_hora para el front: campo sólo lectura

    created_at = serializers.DateTimeField(source='fecha_hora',
read_only=True)

class Meta:

    model = RegistroTag

    fields = ['id', 'tag', 'nombre', 'categoria', 'fecha_hora',
'created_at']

```

Notas de diseño:

- Se introduce `created_at` como **alias** de `fecha_hora` porque el front lo consume con ese nombre.
- `created_at` es **read-only** (la fuente de verdad es `fecha_hora`).

5.4.4 Vistas y ViewSets

El módulo combina:

- **Vistas funcionales** (para hardware/ESP32 donde conviene deshabilitar CSRF), y
- **ViewSets de DRF** (CRUD REST completo para entidades como `Pano1`).

A) Endpoints “simples” (funcionales)



```

from django.http import JsonResponse
from django.views.decorators.http import require_GET
from rest_framework.decorators import api_view, permission_classes
from django.views.decorators.csrf import csrf_exempt, ensure_csrf_cookie
from django.db.models import F
from django.utils.timezone import localtime
from django.utils import timezone
from rest_framework.response import Response
from rest_framework import status
from rest_framework import viewsets, mixins, status
from rest_framework.viewsets import ModelViewSet
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticatedOrReadOnly, AllowAny
from .models import RegistroTag, Pano1, Asignacion
from django.shortcuts import get_object_or_404
from .serializer import Pano1Serializer, RegistroTagSerializer, AsignacionSerializer
import json

@csrf_exempt
def recibir_tag(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            tag = data.get('tag')
            print(tag)
            if tag:
                RegistroTag.objects.create(tag=tag)
                return JsonResponse({'status': 'ok'})
            else:
                return JsonResponse({'error': 'tag faltante'}, status=400)
        except Exception as e:
            return JsonResponse({'error': str(e)}, status=500)

    return JsonResponse({'error': 'método no permitido'}, status=405)

@csrf_exempt
def editar_tag(request, id):
    if request.method == 'PUT':
        tag_obj = get_object_or_404(RegistroTag, pk=id)
        data = json.loads(request.body or "{}")

        if 'nombre' in data:
            tag_obj.nombre = data['nombre']

        if 'categoria' in data:
            raw = data['categoria']
            if raw in (None, '', 'null'):
                tag_obj.categoria = None
            else:
                categoria = str(raw).lower().strip()
                if categoria not in ['persona', 'insumo']:
                    return JsonResponse({

```



```

        'error': 'valor de categoría inválido. Debe ser "persona" o
"insumo".'
    }, status=400)
    tag_obj.categoria = categoria

    tag_obj.save()
    return JsonResponse({
        'status': 'ok',
        'id': tag_obj.id,
        'nombre': tag_obj.nombre,
        'categoria': tag_obj.categoria,
    })
    return JsonResponse({'error': 'Método no permitido'}, status=405)

@ensure_csrf_cookie
def csrf_ping(request):

    return JsonResponse({"detail": "ok"})

@require_GET
def listar_tags(request):
    rows = list(
        RegistroTag.objects
        .order_by("-fecha_hora")
        .values("id", "tag", "nombre", "fecha_hora", "categoria")[:200]
    )
    for r in rows:
        r["created_at"] = localtime(r["fecha_hora"]).strftime("%Y-%m-%d %H:%M:%S")
        del r["fecha_hora"]

    return JsonResponse(rows, safe=False)

@api_view(['POST'])
@permission_classes([AllowAny])
def assignments_auto(request):
    data = request.data or {}
    persona_tag = str(data.get('persona_tag', '')).strip()
    persona_nombre = (data.get('persona_nombre') or '').strip() or None
    items = data.get('items') or []

    if not persona_tag or not items:
        return Response({'error': 'persona_tag e items son obligatorios'}, status=400)

    asignados, devueltos = []

    for it in items:
        item_tag = str((it or {}).get('tag', '')).strip()
        item_nombre = ((it or {}).get('nombre') or '').strip() or None
        if not item_tag:
            continue

```



```

        a = Asignacion.objects.filter(persona_tag=persona_tag, item_tag=item_tag,
        activo=True).first()

        if a:

            a.activo = False
            a.devuelto_en = timezone.now()
            a.save(update_fields=['activo', 'devuelto_en'])
            devueltos.append({
                'id': a.id,
                'item_tag': item_tag,
                'item_nombre': item_nombre or a.item_nombre,
                'devuelto_en': a.devuelto_en,
            })
        else:
            Asignacion.objects.filter(item_tag=item_tag, activo=True).update(
                activo=False,
                devuelto_en=timezone.now()
            )

        a = Asignacion.objects.create(
            persona_tag=persona_tag,
            persona_nombre=persona_nombre,
            item_tag=item_tag,
            item_nombre=item_nombre,
            asignado_en=timezone.now(),
            activo=True
        )
        asignados.append({
            'id': a.id,
            'item_tag': item_tag,
            'item_nombre': item_nombre,
            'asignado_en': a.asignado_en,
        })

    return Response({
        'status': 'ok',
        'persona_tag': persona_tag,
        'persona_nombre': persona_nombre,
        'asignados': asignados,
        'devueltos': devueltos,
        'total_asignados': len(asignados),
        'total_devueltos': len(devueltos),
    }, status=200)

@api_view(['GET'])
@permission_classes([AllowAny])
def assignments_list(request):
    qs = Asignacion.objects.all().order_by('-asignado_en')
    activo = request.GET.get('activo')

```



```

if activo is not None:
    qs = qs.filter(activo=(str(activo).lower() in ['1','true','t','yes','y']))
persona_tag = request.GET.get('persona_tag')
if persona_tag:
    qs = qs.filter(persona_tag=persona_tag)
ser = AsignacionSerializer(qs, many=True)
return Response(ser.data)

@api_view(['PUT'])
@permission_classes([AllowAny])
def assignments_devolver(request, pk: int):
    a = Asignacion.objects.filter(pk=pk, activo=True).first()
    if not a:
        return Response({'error': 'asignación no encontrada o ya devuelta'}, status=404)
    a.activo = False
    a.devuelto_en = timezone.now()
    a.save()
    return Response(AsignacionSerializer(a).data)

class PanolViewSet(ModelViewSet):
    queryset = Panol.objects.all().order_by('nombre')
    serializer_class = PanolSerializer
    permission_classes = [AllowAny]

class RegistroTagViewSet(mixins.CreateModelMixin,
mixins.ListModelMixin, viewsets.GenericViewSet):

    queryset = RegistroTag.objects.all().order_by('-fecha_hora')
    serializer_class = RegistroTagSerializer
    permission_classes = [AllowAny]
    
```

1) Recibir lectura RFID (ESP32)

```
@csrf_exempt
```

```
def recibir_tag(request):
```

```
    """POST JSON {"tag": "..."} desde ESP32."""
```

```
    if request.method == 'POST':
```

```
        try:
```



```

data = json.loads(request.body or "{}")

tag = data.get('tag')

if not tag:

    return JsonResponse({'error': 'tag faltante'},
status=400)

RegistroTag.objects.create(tag=tag)

return JsonResponse({'status': 'ok'})

except Exception as e:

    return JsonResponse({'error': str(e)}, status=500)

return JsonResponse({'error': 'método no permitido'}, status=405)

```

- **CSRF:** exento para que un microcontrolador pueda postear sin headers especiales.
- **Contratos:** espera {"tag": "..."}; responde {"status": "ok"} o {"error": "..."}
- **Códigos HTTP:** 200, 400 (validación), 405 (método), 500 (excepción).

2) Listar últimas lecturas (para la web-app)

```

@require_GET
def listar_tags(request):
    """Últimos 200 registros, con 'created_at' (alias de fecha_hora)."""
    rows = list(
        RegistroTag.objects
        .order_by("-fecha_hora")
        .values("id", "tag", "nombre", "fecha_hora", "categoria")[:200]
    )
    for r in rows:
        r["created_at"] = localtime(r["fecha_hora"]).strftime("%Y-%m-%d
%H:%M:%S")
        del r["fecha_hora"]
    return JsonResponse(rows, safe=False)

```



- Se expone un array JSON ya con `created_at` en formato legible.
- Se limita a **200** para evitar respuestas gigantes.

3) Anotar/editar una lectura

```
@csrf_exempt
def editar_tag(request, id):
    """PUT JSON {"nombre": "...", "categoria": "..."} para anotar un
    tag."""
    if request.method == 'PUT':
        tag_obj = get_object_or_404(RegistroTag, pk=id)
        data = json.loads(request.body or "{}")
        if 'nombre' in data:
            tag_obj.nombre = data['nombre']
        if 'categoria' in data:
            tag_obj.categoria = data['categoria']
        tag_obj.save()
        return JsonResponse({
            'status': 'ok',
            'id': tag_obj.id,
            'nombre': tag_obj.nombre,
            'categoria': tag_obj.categoria,
        })
    return JsonResponse({'error': 'Método no permitido'}, status=405)
```

- Permite **asignar alias** y **categoría** a una lectura específica.
- Respuesta compacta con el estado final del recurso.

4) CSRF “ping” para inicializar cookies (web-app)

```
@ensure_csrf_cookie
def csrf_ping(request):
    """Deja cookie CSRF para la web-app."""
    return JsonResponse({"detail": "ok"})
```



- Útil para Angular/Ionic con `HttpClientXsrfModule` (el navegador guarda cookie y Angular la reenvía en `X-CSRFToken`).

B) ViewSets (DRF)

PanolViewSet (CRUD REST)

```
class PanolViewSet(ModelViewSet):
    """CRUD de pañoles para el menú."""
    queryset = Panol.objects.all().order_by('nombre')
    serializer_class = PanolSerializer
    permission_classes = [AllowAny] # endurecer en producción

    def create(self, request, *args, **kwargs):
        # Evitar duplicados por nombre
        nombre = (request.data.get("nombre") or "").strip()
        if not nombre:
            return Response({"detail": "nombre requerido"},
                            status=status.HTTP_400_BAD_REQUEST)
        icono = request.data.get("icono") or "cube"
        obj, created = Panol.objects.get_or_create(
            nombre=nombre, defaults={"icono": icono}
        )
        ser = self.get_serializer(obj)
        return Response(ser.data,
                        status=status.HTTP_201_CREATED if created else
                        status.HTTP_200_OK)
```

Contratos:

- **GET** `/api/v1/panoles/` → `[{id, nombre, icono}, ...]`
- **POST** `/api/v1/panoles/` body `{"nombre": "...", "icono": "..."} → 201 Created` o `200 OK` si ya existía.
- **PUT/PATCH/DELETE** por id (si el front luego los implementa).

RegistroTagViewSet (opcional DRF)



```
class RegistroTagViewSet(mixins.CreateModelMixin,
                        mixins.ListModelMixin,
                        viewsets.GenericViewSet):
    """Listar/crear registros vía DRF (alternativa a endpoints simples)."""
    queryset = RegistroTag.objects.all().order_by('-fecha_hora')
    serializer_class = RegistroTagSerializer
    permission_classes = [AllowAny]
```

- Alternativa REST “pura” a `recibir_tag/listar_tags`. Útil si el front prefiere un único estilo.

5.4.5 Mapeado de Rutas (URLConf)

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from rest_framework import routers
from .views import recibir_tag, listar_tags, PanolViewSet, csrf_ping, RegistroTagViewSet,
editar_tag, assignments_list, assignments_auto, assignments_devolver

router = DefaultRouter()
router.register(r'panoles', PanolViewSet, basename='panoles')
router.register(r'tags', RegistroTagViewSet, basename='register-tag')

urlpatterns = [
    path('auth/csrf/', csrf_ping, name='csrf-ping'),
    path('', include(router.urls)),
    path('register/tag/', recibir_tag, name="recibir-tag"),
    path('register/tag/list/', listar_tags, name="listar-tags"),
    path('register/tag/<int:id>/editar/', editar_tag, name="editar-tag"),
    path('assignments/', assignments_list, name='assignments_list'),
    path('assignments/auto/', assignments_auto, name='assignments_auto'),
    path('assignments/<int:pk>/devolver/', assignments_devolver,
name='assignments_devolver'),
]

urlpatterns += router.urls
```



Criterio: Se mantiene un estilo **híbrido** por necesidades del hardware (endpoints simples, sin CSRF) y del front (ViewSets DRF limpios).

5.4.6 Formato de datos (contratos)

Lectura RFID — POST [/api/v1/register/tag/](#)

Request

```
{ "tag": "68630808080235D0" }
```

Response

```
{ "status": "ok" }
```

Errores

```
{ "error": "tag faltante" }           // 400
{ "error": "..." }                 // 500
```

Listado de lecturas — GET [/api/v1/register/tag/list/](#)

Response

```
[
  {
    "id": 183,
    "tag": "68630808080235D0",
    "nombre": "Taladro",
    "categoria": "herramienta",
    "created_at": "2025-09-20 10:09:13"
  },
  ...
]
```



Editar una lectura — PUT </api/v1/register/tag/183/editar/>

Request

```
{ "nombre": "Martillo", "categoria": "herramienta" }
```

Response

```
{ "status": "ok", "id": 183, "nombre": "Martillo", "categoria": "herramienta" }
```

Paños — GET/POST </api/v1/panoles/>

POST Request

```
{ "nombre": "Pañol Principal", "icono": "cube" }
```

POST Response (201 o 200 si ya existía)

```
{ "id": 1, "nombre": "Pañol Principal", "icono": "cube" }
```

5.4.7 CSRF y CORS (integración con Angular/Ionic)

- Los endpoints DRF (como </panoles/>) requieren **CSRF** si se usan cookies/sesiones.
- Flujo típico del front:
 1. Llamar a </api/v1/auth/csrf/> para que el servidor envíe la cookie **csrftoken**.



2. Angular, con `HttpClientXsrfModule`, reenvía esa cookie como header `X-CSRFToken` automáticamente.
 3. A partir de ahí, `POST/PUT/DELETE` funcionan sin fricción.
- **CORS:** en `settings.py` se definen los orígenes permitidos del front (ej. `http://localhost:4200`, `http://raspberrypi.local:4200`).

Los **endpoints del ESP32** están **exentos de CSRF** por diseño (`@csrf_exempt`), ya que un microcontrolador no maneja cookies/headers de navegador.

5.4.8 Seguridad y permisos

- En desarrollo, los ViewSets usan `AllowAny` por practicidad.
- En producción, se recomienda:
 - `IsAuthenticated` o `IsAdminUser` según el caso,
 - separar endpoints de **ingesta IoT** (permitidos por IP/red interna o con token precompartido),
 - limitar `editar_tag` a usuarios autenticados/autorizados.

Ejemplo (endurecer panales):

`views.py`

```
from rest_framework.permissions import IsAuthenticated

class PanolViewSet(ModelViewSet):
    permission_classes = [IsAuthenticated]
```

5.4.9 Administración (Django Admin)



registros/admin.py

```
from django.contrib import admin
from .models import Panol, RegistroTag

@admin.register(Panol)
class PanolAdmin(admin.ModelAdmin):
    list_display = ("id", "nombre", "icono")
    search_fields = ("nombre",)

@admin.register(RegistroTag)
class RegistroTagAdmin(admin.ModelAdmin):
    list_display = ("id", "tag", "nombre", "categoria", "fecha_hora")
    search_fields = ("tag", "nombre", "categoria")
    list_filter = ("categoria", "fecha_hora")
```

- Permite verificar en segundos que las lecturas llegan y que los paños existen, sin depender del front.

5.4.10 Extensión prevista (futuro cercano)

- **Modelo Herramienta:** tag (FK opcional a RegistroTag o CharField), nombre, estado, pertenece_a = FK Panol.
- **Modelo Movimiento:** agrupar lecturas cercanas (usuario + herramienta ± ventana de segundos) para inferir retiro/devolución.
- **Reglas de negocio:** alertas por no devolución, bloqueo de herramientas sensibles, trazabilidad por usuario y pañol.
- **Paginación** DRF y filtros (por fecha/tag/categoría/panol).
- **Autenticación** basada en sesiones o JWT para paneles de gestión.

Conclusión del Funcionamiento del Backend:



Con los fragmentos anteriores, el módulo del backend queda:

- **Autodocumentado** (contratos JSON claros),
- **Estable** (versionado, validaciones básicas),
- **Listo para el front** (campos, alias `created_at`, CSRF/CORS),
- **Escalable** (nuevos modelos y viewsets sin romper el existente).

5.5 Aplicación Web (Interfaz de Gestión de Insumos y Usuarios)

La capa de interfaz de SICAP se implementa como una SPA con **Ionic + Angular** y consume la API REST de Django. Este apartado documenta la arquitectura, el flujo entre pantallas (**Login** → **Menú de usuarios** → **Home por usuario**), el manejo de cookies/CSRF para operaciones de escritura, y los fragmentos esenciales de código que materializan cada función.

5.5.1 Tecnologías y organización

Stack UI

- **Ionic Framework** (componentes móviles responsivos).
- **Angular** (enrutado, módulos, DI, `HttpClient`).
- **TypeScript** (lógica de componentes/servicios).
- **HTML5 + SCSS** (maquetación, estilos).
- **RxJS** (polling y composición de flujos).

Estructura relevante

```
src/
  app/
    login/      # Pantalla de autenticación
    menu/      # Selección/creación de usuarios (perfiles)
    home/      # Panel por usuario, con secciones internas
```



```
app-routing.module.ts
app.module.ts
assets/           # Imágenes, logos, animación MP4/Lottie
environments/
  environment.ts   # API_BASE (dev)
  environment.prod.ts # API_BASE (prod)
global.scss
```

Configuración de entorno (única fuente de verdad para la API)

```
// environments/environment.ts
```

```
export const environment = {
  production: false,
  API_BASE: 'http://raspberrypi.local:5000/api/v1/' // terminar SIEMPRE
  con /
};
```

Módulos Angular utilizados

- `HttpClientModule` para llamadas a la API.
- `FormsModule` / `IonicModule` para formularios y UI.

5.5.2 Login (autenticación inicial)

Objetivo. Validar credenciales contra el backend y, en caso de éxito, navegar al Menú de usuarios. En esta versión no se persiste token; la verificación es stateless (se procesa la respuesta y se redirige).

Interfaz. Formulario minimalista con 2 campos (`username`, `password`) y botón de envío. Se aplican estilos de Ionic y, opcionalmente, una animación de fondo (Lottie o video MP4 en `assets/`).

Flujo.



1. El usuario completa el formulario.
2. `POST /usuarios/login/` con `{ username, password }`.
3. Si la respuesta es `{ success: true }`, se navega a `/menu`.

Fragmentos esenciales

HTML:

```
<ion-header [translucent]="true" *ngIf="animacionTerminada">
</ion-header>

<ion-content [fullscreen]="true">

  <div *ngIf="!animacionTerminada" class="splash-container">
    <lottie-player
      src="assets/Animaciones/animacion.login.json"
      background="transparent"
      speed="1"
      autoplay
      (complete)="onAnimationComplete()">
    </lottie-player>
  </div>

  <div *ngIf="animacionTerminada" class="login-content ion-padding">

    <div class="logo-container">
      
    </div>

    <ion-card>
      <ion-card-content>
        <form (ngSubmit)="login()">
          <ion-item>
            <ion-icon name="person-outline" slot="start" color="medium"></ion-icon>
            <ion-label position="floating">Usuario</ion-label>
            <ion-input type="text" [(ngModel)]="username" name="username"></ion-input>
          </ion-item>

          <ion-item>
            <ion-icon name="lock-closed-outline" slot="start" color="medium"></ion-icon>
            <ion-label position="floating">Contraseña</ion-label>
            <ion-input type="password" [(ngModel)]="password" name="password"></ion-input>
          </ion-item>
        </form>
      </ion-card-content>
    </ion-card>
  </div>
</ion-content>
```



```

        <br>

        <ion-button type="submit" expand="block" shape="round">
            Ingresar
        </ion-button>
    </form>
</ion-card-content>
</ion-card>
</div>

</ion-content>

TypeScript:
import { Component } from '@angular/core';

import { NavController, AlertController, LoadingController } from '@ionic/angular';

@Component({
  selector: 'app-login',
  templateUrl: './login.page.html',
  styleUrls: ['./login.page.scss'],
  standalone: false,
})
export class LoginPage {

  animacionTerminada: boolean = false;
  username = '';
  password = '';

  constructor(
    private navCtrl: NavController,
    private alertCtrl: AlertController,
    private loadingCtrl: LoadingController
  ) { }

  onAnimationComplete() {
    this.animacionTerminada = true;
  }

  async login() {

    const loading = await this.loadingCtrl.create({
      message: 'Verificando...',
    });
    await loading.present();

    try {
      const response = await fetch('http://192.168.111.218:5000/usuarios/login/', {
        method: 'POST',
      });
    }
  }
}

```



```

    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: this.username, password: this.password })
  });

  await loading.dismiss();

  if (!response.ok) {

    throw new Error('Error en el servidor');
  }

  const data = await response.json();

  if (data.success) {
    this.navCtrl.navigateForward('/menu');
  } else {

    this.mostrarAlerta('Error de Login', data.message || 'Credenciales incorrectas.');

```

Notas técnicas.

- `withCredentials: true` se deja habilitado desde el comienzo para mantener coherencia con endpoints que sí requieren cookie/CSRF.
- Si en el futuro se añade sesión basada en cookies o JWT, el punto de integración permanece encapsulado en este mismo método.

5.5.3 Menú de usuarios (creación y selección)



El Menú lista los **usuarios/perfiles** (modelo *Pañol* en el backend) y permite **crear** uno nuevo. La pantalla emula un “selector Netflix”: una tarjeta para “**Crear Usuario**” y tarjetas para cada perfil existente. Al seleccionar un perfil, se navega a su **Home**.

Requisitos de seguridad. Las operaciones de **creación** requieren **CSRF**:

- Antes de operar, se hace `GET /api/v1/auth/csrf/` para **plantar** la cookie `csrftoken`.
- Peticiones subsecuentes que incluyan `{ withCredentials: true }` enviarán `X-CSRFToken` automáticamente (con CORS y CSRF confiados correctamente configurados en el backend).

Flujo.

1. Al inicializar la página: `GET /auth/csrf/` y luego `GET /panoles/`.
2. Creación: `POST /panoles/` con `{ nombre, icono }`.
3. Navegación: `router.navigate(['/home', id])`.

Fragmentos esenciales

TypeScript:

```
import { Component, OnInit } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { environment } from 'src/environments/environment';
import { Router } from '@angular/router';

type Panol = { id?: number; nombre: string; icono?: string };

@Component({
  selector: 'app-menu',
  templateUrl: './menu.page.html',
  styleUrls: ['./menu.page.scss'],
  standalone: false,
})
export class MenuPage implements OnInit {
  private API_BASE = environment.API_BASE;
  private PANOLES_URL = `${this.API_BASE}panoles/`;
  private CSRF_PING_URL = `${this.API_BASE}auth/csrf/`;

  panoles: Panol[] = [];
  loading = false;
}
```



```
error?: string;

creating = false;
nombreNuevo = '';
saving = false;

constructor(private http: HttpClient, private router: Router) {}

ngOnInit() {
  // 1) Plantar cookie CSRF y luego cargar pañoles
  this.http.get(this.CSRF_PING_URL, { withCredentials: true }).subscribe({
    next: () => this.cargar(),
    error: () => this.cargar(), // aunque falle, intento cargar
  });
}

cargar() {
  this.loading = true;
  this.error = undefined;

  console.log('👉 PANOLES_URL =', this.PANOLES_URL);

  this.http.get<any>(this.PANOLES_URL, { withCredentials: true }).subscribe({
    next: (res) => {
      this.panoles = Array.isArray(res) ? res : (res?.results ?? []);
      this.loading = false;
    },
    error: () => {
      this.error = 'No pude cargar los pañoles';
      this.loading = false;
    },
  });
}

abrirCrear() {
  this.nombreNuevo = '';
  this.creating = true;
}

cancelarCrear() {
  this.creating = false;
  this.nombreNuevo = '';
}

crear() {
  const nombre = this.nombreNuevo.trim();
  if (!nombre) return;

  this.saving = true;

  this.http
    .post<Panol>(
```



```

        this.PANOLES_URL,
        { nombre, icono: 'cube' },
        { withCredentials: true }
    )
    .subscribe({
      next: (p) => {
        this.saving = false;
        this.creating = false;
        this.nombreNuevo = '';
        // lo muestro al toque
        this.panoles.unshift(p);
      },
      error: () => {
        this.saving = false;
        this.error = 'No pude crear el pañol';
      },
    });
  }

  abrirPanol(p: Panol) {
    if (!p.id) return;
    this.router.navigate(['/home', p.id]);
  }
}

```

HTML:

```

<ion-content [fullscreen]="true" class="page">
  <div class="wrap">

    <div class="center" *ngIf="loading"><ion-spinner></ion-spinner></div>
    <ion-text color="danger" class="center" *ngIf="!loading && error">{{ error }}</ion-text>

    <section class="grid" *ngIf="!loading">

      <article class="card new" (click)="abrirCrear()">
        <div class="poster plus"></div>
        <div class="name">Crear Usuario</div>
      </article>

      <article class="card" *ngFor="let p of panoles">
        <button class="poster" (click)="abrirPanol(p)">
          <ion-icon [name]=" 'cube' "></ion-icon>
        </button>
        <div class="name">{{ p.nombre }}</div>
      </article>
    </section>

    <ion-modal mode="md" cssClass="tarjeta-modal" [isOpen]="creating"
      (didDismiss)="cancelarCrear()" [presentingElement]="null">
      <ng-template>

```



```

<ion-header>
  <ion-toolbar>
    <ion-title>Personalizar</ion-title>
    <ion-buttons slot="end">
      <ion-button (click)="cancelarCrear()" class="btn-close">
        <ion-icon name="close-outline"></ion-icon>
      </ion-button>
    </ion-buttons>
  </ion-toolbar>
</ion-header>
<ion-content class="ion-padding">
  <ion-item>
    <ion-input label="Nombre" labelPlacement="floating"
[(ngModel)]="nombreNuevo"></ion-input>
  </ion-item>
  <div class="row mt">
    <ion-button [disabled]="saving || !nombreNuevo.trim()"
(click)="crear()">Crear</ion-button>
    <ion-button fill="clear" (click)="cancelarCrear()">Cancelar</ion-button>
  </div>
  <ion-text color="danger" *ngIf="error">{{ error }}</ion-text>
</ion-content>
</ng-template>
</ion-modal>

</div>
</ion-content>

```

El diseño de cuadrícula se resuelve con SCSS local (tarjetas, poster circular/ícono, hover, etc.), manteniendo la estética “selector de perfiles”.

5.5.4 Home por usuario (secciones internas)

El **Home** se instancia por ruta (`/home/:panoId`) y concentra cuatro secciones:

- **Interfaz Visual** (clasificación en tiempo real de tags “En Taller” vs “Fuera del Taller”).
- **Inventario** (vista única por tag con nombre/categoría).
- **Empleados** (listado orientado a personas).



- **Personalización** (reservado para ajustes).

Orígenes de datos.

- `GET /api/v1/register/tag/list/` entrega los últimos registros en JSON.
- `PUT /api/v1/register/tag/{id}/editar/` permite editar **nombre** y **categoría** de un registro.

Animación de fondo. Se utiliza un video MP4 (`assets/Animaciones/animacion_sicap.mp4`) en bucle, con superposición (`.bg-overlay`) para fortalecer contraste del contenido.

Interfaz Visual (lógica).

- Se mantiene en memoria un diccionario `tagsEstado: { [tag]: 'taller' | 'fuera' }`.
- Por **cada nuevo registro** de un tag:
 - Si el tag no existe aún → se coloca en **En Taller**.
 - Si estaba **En Taller** → pasa a **Fuera del Taller**.
 - Si estaba **Fuera del Taller** → regresa a **En Taller**.
- La sección se actualiza cada **3 segundos** con `RxJS interval`.

Fragmentos (procesamiento y polling):

`// home.page.ts (fragmento)`

```
const BASE = environment.API_BASE.replace(/\/$/, '');
const API_LIST = `${BASE}/register/tag/list`;
const API_EDIT = `${BASE}/register/tag`;

activarSeccion(nombre: string) {
  this.seccionActiva = nombre;
  if (nombre === 'Interfaz Visual') {
    this.cargarRegistros();
  }
}
```



```

    this.sub = interval(3000).subscribe(() => this.cargarRegistros());
  } else {
    this.sub?.unsubscribe();
  }
}

private procesarRegistro(nuevo: Registro) {
  const tag = nuevo.tag;
  const estado = this.tagsEstado[tag]; // 'taller' | 'fuera' |
  undefined
  if (!estado) { this.tagsTaller.unshift(nuevo); this.tagsEstado[tag] =
'taller'; return; }
  if (estado === 'taller') {
    this.tagsTaller = this.tagsTaller.filter(t => t.tag !== tag);
    this.tagsFuera.unshift(nuevo);
    this.tagsEstado[tag] = 'fuera';
  } else {
    this.tagsFuera = this.tagsFuera.filter(t => t.tag !== tag);
    this.tagsTaller.unshift(nuevo);
    this.tagsEstado[tag] = 'taller';
  }
}

cargarRegistros() {
  this.http.get<Registro[]>(API_LIST).subscribe(rows => {
    this.registros = rows;
    rows.forEach(reg => {
      const yaClasificado = this.tagsTaller.some(t => t.id === reg.id) ||
this.tagsFuera.some(t => t.id === reg.id);
      if (!yaClasificado) this.procesarRegistro(reg);
    });
  });
}

```

Inventario (tags únicos).

Se deriva una colección **sin duplicados por tag**, conservando el **más reciente**:

// home.page.ts (fragmento)

```

get tagsUnicos(): Registro[] {

```



```
const vistos = new Set<string>();
const unicos: Registro[] = [];
for (let i = this.registros.length - 1; i >= 0; i--) {
    const r = this.registros[i];
    if (!vistos.has(r.tag)) { vistos.add(r.tag); unicos.unshift(r); }
}
return unicos;
}
```

Edición inline (nombre/categoría).

Desde **Inventario** o **Empleados** se abre un **ion-alert** con inputs y se envía un **PUT**:

// home.page.ts (fragmento)

```
editarTag(reg: Registro, nombre: string, categoria: string) {
    this.http.put(`${API_EDIT}/${reg.id}/editar/`, { nombre, categoria })
        .subscribe(() => { reg.nombre = nombre; reg.categoria = categoria; });
}
```

Estructura HTML (extractos clave).

Barra lateral para secciones + contenedor de tarjetas por columna:

```
<!-- home.page.html (fragmentos) -->
<div class="vertical-toolbar" [class.activa]="seccionActiva">
    <ion-button *ngFor="let opcion of opcionesMenu"
        [disabled]="opcion === seccionActiva"
        (click)="activarSeccion(opcion)">{{ opcion }}</ion-button>
    <ion-button color="medium"
        (click)="volverAlMenuCentral()">Volver</ion-button>
</div>

<!-- Interfaz Visual: dos columnas -->
<div class="cajas-tags" *ngIf="seccionActiva === 'Interfaz Visual'">
    <div class="caja-tags">
        <h3>En Taller</h3>
        <ion-card *ngFor="let r of tagsTaller">...</ion-card>
    </div>
    <div class="caja-tags">
```



```
<h3>Fuera del Taller</h3>
<ion-card *ngFor="let r of tagsFuera">...</ion-card>
</div>
</div>
```

5.5.5 Enrutado

El ruteo conecta las tres pantallas principales y permite instanciar el **Home** por identificador de usuario:

// app-routing.module.ts (fragmento)

```
const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', loadChildren: () =>
import('./login/login.module').then(m => m.LoginPageModule) },
  { path: 'menu', loadChildren: () => import('./menu/menu.module').then(m
=> m.MenuPageModule) },
  { path: 'home/:panoId', loadChildren: () =>
import('./home/home.module').then(m => m.HomePageModule) },
];
```

5.5.6 Estilos y composición visual

Video de fondo

El Home aplica un video en loop con superposición semitransparente para mejorar la legibilidad de tarjetas/labels:

```
/* home.page.scss (fragmento) */
.has-video-bg {
  #bg-video {
    position: fixed; inset: 0; width: 100%; height: 100%;
    object-fit: cover; z-index: 0;
```



```

}
.bg-overlay {
  position: fixed; inset: 0;
  background: rgba(0,0,0,.35);
  z-index: 1;
}
.vertical-toolbar, .visual-content, .botones-menu-central { z-index: 2;
position: relative; }
}

```

Barra lateral y área de trabajo

```

/* home.page.scss (fragmento) */
.vertical-toolbar {
  position: fixed; top: 56px; left: 0; width: 220px;
  height: calc(100vh - 56px); padding: 10px; background: #888;
  display: flex; flex-direction: column; gap: 8px;
}
.visual-content { margin-left: 220px; padding: 20px; }
.cajas-tags { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }

```

Grid del Menú (selector de usuarios)

```

/* menu.page.scss (fragmento) */
.grid { display: grid; grid-template-columns: repeat(auto-fill,
minmax(160px,1fr)); gap: 16px; }
.card { display: grid; gap: 8px; text-align: center; }
.poster { aspect-ratio: 1/1; border-radius: 12px; display: grid;
place-items: center; }
.plus { font-size: 48px; }

```

5.5.7 Cookies, CSRF y CORS (desde el frontend)



- **CSRF:** previo a **POST/PUT/DELETE**, se realiza **GET /api/v1/auth/csrf/** para que el backend emita la cookie **csrftoken**.
- **withCredentials:** todas las llamadas que mutan estado se envían con **{ withCredentials: true }** para que el navegador remita cookies y Angular agregue **X-CSRFToken**.
- **CORS:** el backend publica **CORS_ALLOWED_ORIGINS** y **CSRF_TRUSTED_ORIGINS** coherentes con el host del frontend. No se requiere configuración adicional en el cliente.

Resumen funcional

1. **Login** valida credenciales y redirige a **Menú**.
2. **Menú** planta CSRF, lista usuarios y permite crear nuevos.
3. **Home** por usuario consume registros, actualiza la **Interfaz Visual** en tiempo real, muestra **Inventario** de tags únicos y permite **edición inline** (nombre/categoría).

La combinación de Angular (estructura y datos), Ionic (UI) y el contrato de API de Django (JSON simple con CSRF/credenciales) deja un frontend **cohesionado, escalable y mantenible**.

6. Base de Datos: Modelo Relacional, Migraciones y Pruebas

La base de datos de SICAP es el componente que almacena de forma permanente toda la información relevante del sistema: principalmente los registros de movimientos (lecturas RFID), así como datos de usuarios y potencialmente herramientas. Al ser un proyecto de alcance local y relativamente pequeño, se optó por un **modelo de base de datos relacional** sencillo usando SQLite, que viene integrado con Django.

Modelo de datos (relacional): A continuación se describe el esquema de tablas más importante. Para mejor visualización, se presenta en formato de tabla:

- **registros_registrotag** – (Tabla de los registros de tags leídos)



Columnas:

- *id* (Integer, PK, auto-incremental)
- *tag* (Text) – Almacena el identificador del tag RFID leído. Puede contener letras y números (en formato hex u otro). Ejemplos de valores: “4B01020304AABBCC”.
- *fecha_hora* (DateTime) – Marca temporal de cuándo se insertó el registro. Ej: “2025-10-03 14:22:10”.

Descripción: Cada fila representa una detección de una etiqueta RFID en el portal, registrada automáticamente al recibirse del ESP32. La combinación de *tag* + *fecha_hora* indica qué tag fue detectado y en qué momento exacto.

- **auth_user** – (Tabla de usuarios, estándar de Django)

Columnas principales: id, username, password (hashed), email, first_name, last_name, is_staff, is_superuser, etc.

Descripción: Almacena las cuentas de usuario para acceder al sistema web. Por defecto Django crea esta tabla. En nuestro caso, contendrá por ejemplo: usuario “admin” (superuser), y cualquier otro creado (p. ej. “juan”, “maria” para operarios con acceso a la web). Contraseñas en hash (no visibles). El campo is_staff se usa para permitir acceso al /admin; is_superuser para todos permisos.

- **usuarios_login_pañol** – (Tabla de pañoles administrados)

Columnas:

- *id* (PK)
- *nombre* (varchar(100)) – Nombre del pañol, ej: “Pañol Electrónica”.
- *usuario_id* (FK -> auth_user.id) – Referencia al usuario encargado de ese pañol.

Descripción: Si la institución tuviera múltiples pañoles (herramientas, insumos, biblioteca), esta tabla los lista y vincula cada uno a un usuario responsable. En el prototipo se podría tener una fila: nombre=“Pañol Taller”, usuario_id = (id de admin). Esto no se usó ampliamente en la lógica, pero sienta base para expansión multi-área.

- **usuarios_login_programmer** – (Tabla de programadores del proyecto, auxiliar)

Columnas: id, fullname (varchar(100)).



Descripción: Lista miembros del equipo de desarrollo. Este modelo no interactúa con el funcionamiento de SICAP en producción; fue más bien utilizado para registrar a los desarrolladores como parte del proyecto escolar. Se puede ignorar para replicación (no afecta funcionalidades).

- (Posibles tablas ausentes en esta versión):

Si se extendiera el modelo, podrían agregarse:

- *herramientas_herramienta* – con campos id, tag, nombre, descripcion, estado, etc.
- *operarios_usuario* – lista de personas que usan herramientas, si se quieren separar de la tabla auth_user (o se podría reutilizar auth_user con grupos/roles).
- *prestamos_prestamo* – tabla relacional que vincule herramienta, usuario, fecha_salida, fecha_entrada, etc., de modo explícito en vez de deducir de registros.

En la versión actual, estos no existen; en su lugar, la lógica se infiere de registros.

Migraciones: Django maneja las migraciones de esquema. En el repositorio, en registros/migrations/ se encuentra, por ejemplo, 0001_initial.py (creada al hacer makemigrations) que define la creación de la tabla RegistroTag. También hay 0002_rename_tag_id_registrotag_tag.py – esto sugiere que inicialmente el campo se llamaba tag_id y luego se renombró a tag. Efectivamente, el README menciona tag_id pero en código final quedó tag. Esta migración renombra la columna en la BD.

Al replicar, si se ejecuta migrate, Django aplicará directamente la última estructura (se saltará rename porque será primera vez). Las migraciones se incluyen más por integridad que por necesidad para un nuevo entorno.

Integridad de datos: Actualmente no hay restricciones de clave externa entre RegistroTag y otras tablas, porque guardamos solo el tag string. Esto significa que la integridad referencial (por ejemplo, asegurar que un tag corresponde a una herramienta existente) recae en la capa de aplicación, no en la BD. Para replicar el funcionamiento tal cual, esto no es un problema. Sin embargo, al ampliar, sería recomendable asociar RegistroTag con, por ejemplo, un modelo Herramienta para evitar tags desconocidos. También se podría establecer relación de RegistroTag con User (para registrar quién sacó qué, si el sistema detectara el usuario tag en mismo evento).

Consultas típicas:

- *Últimos movimientos:* SELECT tag, fecha_hora FROM registros_registrotag ORDER BY fecha_hora DESC LIMIT 10; – Muestra los 10 eventos más recientes.



- *Filtrar por tag*: `SELECT * FROM registros_registrotag WHERE tag = "ABC123" ORDER BY fecha_hora`; – ver histórico de un tag en particular (útil para ver cuántas veces se usó una herramienta).
- *Usuarios*: `SELECT username, id FROM auth_user`; – lista de usuarios del sistema web.
- *Combinar info*: Dado que no hay foreign keys directas, una consulta para saber qué usuario tenía tal herramienta en cierto momento requiere deducción. Se podría correlacionar: “buscar registro de usuario X y herramienta Y con timestamps cercanos”. Esto en SQL puro es complejo; es más fácil procesarlo en Python tras obtener listas.

Pruebas de la base de datos: Durante el desarrollo, se realizaron algunas pruebas para asegurar que los modelos funcionaban:

- Tras migrar, se usó la *shell de Django* (`python manage.py shell`) para crear manualmente un `RegistroTag`:

```
from registros.models import RegistroTag
r = RegistroTag.objects.create(tag="TESTTAG123")
print(r.id, r.fecha_hora)
```

- Esto confirmó que se guardaba con timestamp actual.
- También se probó leer los registros:

```
for reg in RegistroTag.objects.all():
    print(reg.tag, reg.fecha_hora)
```

- Esto mostró en consola los que se habían guardado (vía pruebas con curl, etc.).
- Se ingresaron datos de prueba para usuarios (`createsuperuser` ya crea uno).
Adicionalmente:

```
from django.contrib.auth.models import User
u = User.objects.create_user('juan', password='1234')
```

- Este usuario luego se probó con la API login (devolviendo `success True` con contraseñas correctas).
- Integridad: como no hay muchas restricciones, la principal preocupación era no tener duplicados no deseados. Se consideró si debíamos evitar registrar el mismo tag repetido en milisegundos. Ese control se puso en ESP32. En BD no hay restricción



unique sobre tag, porque se espera repeticiones a distintos tiempos.

Scripts de prueba automatizados: No se llegaron a integrar scripts de prueba completos, pero se podría crear un script Python para simular la entrada de datos:

```
import requests, time
tags = ["AAA111", "BBB222"]
for t in tags:
    requests.post("http://192.168.111.218:8000/api/v1/register/tag/", json={"tag": t})
```

Primero envía AAA111 (supongamos es user) luego BBB222 (herramienta). Repetir en orden inverso para simular devolución:

```
time.sleep(5)
for t in reversed(tags):
    requests.post(..., json={"tag": t})
```

Luego consultar base de datos a ver si hay 4 nuevos registros. Esto fue mentalmente verificado más que codificado.

Respaldo y administración: La BD SQLite es un archivo db.sqlite3 en el directorio del proyecto. Es importante hacer respaldos regulares, sobre todo antes de actualizaciones. Se puede copiar directamente el archivo (con el servidor apagado para consistencia). También se puede usar python manage.py dumpdata > backup.json para exportar datos en JSON si se desea migrar a otra BD.

Para la presentación del proyecto, se podría pre-cargar la BD con algunos datos ficticios para demostrar la interfaz, por ejemplo, unos registros de ejemplo, usuarios de prueba, etc.

Conclusión de BD: La base de datos fue diseñada minimalista pero funcional. Para replicar el proyecto, bastará con ejecutar las migraciones en la nueva instalación, lo cual creará el esquema adecuado. Dado el bajo nivel de complejidad de relaciones en este esquema, es poco propenso a errores de integridad y sencillo de entender para cualquier persona que tome el proyecto. A la vez, es fácil de extender en el futuro conforme se requiera persistir más información (por ejemplo, agregar campos de ubicación de herramienta, estado de mantenimiento, etc.).

7. Capturas de Pantalla, Esquemas y Fotografías del Sistema

En esta sección se presentan algunas imágenes ilustrativas del proyecto SICAP, incluyendo esquemas de red, pantallas de la aplicación y fotografías de componentes físicos, para complementar la descripción técnica.



Figura 3: Esquema general del sistema SICAP (arquitectura de red y módulos). El ESP32 con lector RFID se comunica vía WiFi con el servidor Raspberry Pi a través del router dedicado. El servidor almacena los datos en la base de datos local y provee la interfaz web de consulta. Los usuarios administrativos pueden acceder a la aplicación web desde laptops o smartphones conectados a la red interna.

*(Nota: El diagrama simplificado anterior conceptualiza la arquitectura, mostrando la interacción entre **Portal RFID (ESP32)**, **Router WiFi** y **Servidor (Raspberry Pi)** con la **Base de Datos** y la **Web App**.)*

Figura 4: Conversor RS232 a TTL desarrollado para SICAP. Este pequeño circuito impreso permite conectar el módulo lector RFID de interfaz serial RS232 al microcontrolador ESP32. Incluye el integrado MAX3232 (centro) y sus capacitores asociados, así como un conector DB9 (derecha) para el cable del lector. Mediante este conversor, los datos de las etiquetas RFID leídos por el módulo pueden ser interpretados correctamente por el ESP32 sin dañar sus puertos.

En la fotografía de la Figura 4 se aprecia el cuidado en la elaboración artesanal de la PCB y la simpleza del circuito: 4 condensadores para el MAX3232 y los pines de cabecera para 5V, GND, TX, RX hacia el ESP32. Este componente, aunque sencillo, fue crucial para la integración del hardware RFID.

Capturas de la Aplicación Web: A continuación, se muestran pantallas de la interfaz web de SICAP en su fase prototipo:

- *Pantalla de Login:* El diseño presenta un fondo alusivo con el logo de SICAP y campos para usuario y contraseña. Un botón “Iniciar Sesión” permite validar las credenciales. Esta es la puerta de entrada segura al sistema de gestión.
- *Panel Principal:* Luego de ingresar, el usuario ve un menú de opciones con botones grandes y descriptivos (Interfaz Visual, Registros y Movimientos, Reportes, Gestión de Pañol, etc.). Desde aquí navega a la funcionalidad deseada. El diseño es minimalista y responsivo, adaptándose tanto a monitores como a pantallas de tablets/móviles mediante Ionic.
- *Vista de Movimientos (prototipo):* Muestra una tabla con columnas como *Etiqueta*, *Fecha/Hora*, *Descripción* (ej. “Retirada” o “Devuelta”) y posiblemente *Usuario*. En el estado actual, esta información se derivaría de los registros en BD. Idealmente se mostraría ya el nombre de la herramienta en vez del código de tag (eso requeriría vincular el tag con una descripción almacenada). A pesar de eso, incluso ver los códigos y timestamps brinda al administrador la información cruda necesaria.



(Nota: Debido a la naturaleza del prototipo y a que la interfaz está en desarrollo, se omiten algunas capturas dado que muchas se encuentran en estado de diseño más que funcional. Sin embargo, la estructura para presentarlas está implementada tal como se describe.)

Fotografías del sistema en funcionamiento:

- *Portal RFID instalado:* En la Figura 1 previamente insertada (Figura 1, sección 4) se muestra el diseño del portal. En la realidad, este portal se construyó con listones de madera de 2 metros de alto, con las antenas UHF ubicadas a ambos lados a aproximadamente 1.2 m del suelo (a esa altura se optimizó la lectura para herramientas llevadas en mano o en una caja).
- Se hicieron pruebas pasando objetos con etiquetas a distintas velocidades. El sistema detectó herramientas hasta a ~2 metros de la antena en pruebas de campo, confirmando la elección de RFID UHF. Una persona pasando rápidamente (paso normal) por el arco con varias herramientas logró lecturas exitosas de todos los tags en la mayoría de intentos, gracias a la capacidad multi-tag del estándar UHF.
- *Montaje del hardware en el pañol:* El ESP32 y el conversor RS232-TTL se alojaron dentro de una pequeña caja plástica en la parte superior del marco del portal, junto con el módulo lector. De esa caja salía el cable de la antena hacia los paneles. Se prefirió tener cables cortos TTL entre ESP32 y conversor, manteniendo el RS232 en cable más largo hacia el lector si era necesario.
- La Raspberry Pi se ubicó en la oficina del encargado, conectada por WiFi (aunque se contempló cablear). Junto a ella, un monitor exhibía la interfaz web (mostrando en tiempo real cómo aparecen registros cuando alguien pasaba por el portal, lo cual fue muy útil en la demo final).
- *Etiqueta RFID en herramienta:* Se utilizaron tags UHF autoadhesivos diseñados para superficies metálicas (on-metal), de aproximadamente 5x5 cm, con capa aislante. Se pegaron en las herramientas en lugares planos. Una foto de ejemplo mostraba un destornillador grande con el tag adherido en el mango, y otra con un taladro de mano con el tag en un costado. Estas etiquetas fueron proveídas por un sponsor y tenían impresos códigos QR además del chip RFID, pensando en posible doble lectura (no implementada, pero interesante para backup manual si falla RFID).
- *Pulsera RFID para usuario:* Consistía en una pulsera plástica con un módulo RFID encapsulado (similar a las usadas en natatorios/gimnasios para lockers). Se programaron IDs únicas en ellas. La comodidad de la pulsera es que el operario no tiene que sacar nada de su bolsillo, simplemente al pasar por el portal será detectado. En las pruebas, esto funcionó siempre que el usuario levantara ligeramente el brazo al altura del pecho al pasar (para quedar en el lóbulo de lectura de la antena).



Esquema de red y flujo de datos: (Referido en Figura 3) Se reitera el concepto del flujo ya mencionado pero apoyado visualmente:

- Flecha desde *Herramienta con tag* y *Usuario con tag* hacia *Antena/ESP32* (lectura RFID inalámbrica).
- Flecha desde *ESP32* hacia *Router* (comunicación WiFi TCP/IP).
- Flecha desde *Router* hacia *Raspberry Pi* (paquete IP entrante al puerto 8000).
- Flecha interna en Pi desde *Django* hacia *SQLite* (operación de escritura de registro).
- Finalmente, flechas desde Pi hacia *Web Browser* de *Admin* (cuando consulta vía HTTP, lee desde SQLite y envía datos a la web).

Cada flecha representa tecnologías distintas: RFID (radio), WiFi (802.11), HTTP/REST, y consulta de base de datos SQL, demostrando la variedad de capas integradas en SICAP.

Estas imágenes y descripciones proporcionan una idea más tangible de cómo es SICAP tanto en su interfaz virtual como en su implementación física. En una presentación final, sería recomendable contar con:

- **Capturas de la interfaz web** demostrando un caso de uso (por ejemplo, una pantalla mostrando “Herramienta X falta, última vista por Usuario Y a tal hora”).
- **Fotos del montaje real** en el taller (portal instalado en una puerta, etc.).
- **Diagramas técnicos** que ya hemos incluido, para que los lectores de la carpeta técnica visualicen el sistema de un vistazo.

(En ausencia de incluir esas imágenes aquí por limitaciones de formato, se sugiere agregarlas en la versión imprimible de la carpeta técnica).

8. Pruebas Realizadas y Resultados Esperados

La calidad y confiabilidad de SICAP se aseguraron mediante una serie de **pruebas integrales**, abarcando desde pruebas unitarias de módulos individuales hasta pruebas de sistema con todos los componentes funcionando conjuntamente. A continuación se describen las principales pruebas realizadas, los escenarios cubiertos y los resultados observados, así como los resultados esperados para el sistema en un entorno real.

9.1 Pruebas unitarias y de integración de componentes:



- **Prueba del módulo RFID (ESP32 + Lector):**

- *Objetivo:* Verificar que el ESP32 pueda leer correctamente etiquetas RFID a través del módulo lector y conversor.
- *Método:* Se conectó solo el ESP32 con el lector (sin enviar a servidor) y se abrió la consola serial. Se pasó una etiqueta conocida por la antena.
- *Resultado:* El ESP32 imprimió el código hex esperado de 8 bytes, coincidente con la ID de la etiqueta (validado contra el número de serie del tag). Se probó con 5 etiquetas distintas, todos los códigos fueron capturados correctamente.
- *Se midió* el tiempo entre lecturas: el lector envía inmediatamente al detectar un tag, y con el ESP32 en bucle continuo, se lograron lecturas con intervalos inferiores a 100 ms cuando se presentaban tags consecutivos (lo suficientemente rápido).
- *También* se probó alejar la etiqueta lentamente hasta perder lectura: aproximadamente a >3 metros ya no la detectaba, lo cual define el rango práctico ~2.5 m (lo esperado dado las especificaciones de las antenas).

- **Prueba de conexión WiFi y envío HTTP (ESP32→Pi):**

- *Objetivo:* Asegurar que el ESP32 pudiera enviar datos a la Raspberry Pi vía WiFi.
- *Método:* Antes de integrar con RFID, se cargó un script de prueba que cada 10 segundos hacía un POST con un mensaje fijo ("test":"ok") al servidor (utilizando un sencillo view eco en Django).
- *Resultado:* El servidor recibió los POST según lo visto en los logs, y el ESP32 imprimió que recibió respuesta 200 OK. La latencia medida (usando time_ticks_ms antes y después del urequests) fue de ~50 ms por petición en promedio dentro de la LAN, excelente.
- *Se probó desconexión:* Apagando el router por un momento y volviéndolo a encender. El ESP32 detectó pérdida de WiFi (wlan.isconnected false) e intentó reconectar en loop (dado que nuestro código no lo implementaba, se quedó esperando; esto reveló la importancia de manejar reconexiones). Al reencender router, se tuvo que resetear manualmente el ESP32 en esta prueba. Conclusión: implementar reintento de WiFi en intervals es deseable.
- *Simulación de servidor caído:* Se apagó el servidor Django mientras ESP32 enviaba. El ESP32 arrojó excepción en urequests (Connection error).



Capturamos esa excepción y lo hace imprimir error pero seguir corriendo. Cuando se levantó de nuevo el server, siguió normal. Esto confirma robustez ante fallos temporales del servidor (no crashea el ESP).

- **Prueba del backend Django (unidad):**

- Se ejecutaron tests manuales a la vista recibir_tag con distintas entradas:
 - JSON válido con tag => Debe crear objeto. Efectivamente, tras POST manual, en admin se vio el nuevo RegistroTag.
 - JSON mal formado => El servidor respondió status 500 con error de JSON parse, como manejamos except general, respondió {'error': '...'} 500. Lo ideal sería rechazar con 400, pero dependía del error. Esto se consideró aceptable dado que en condiciones normales el ESP32 siempre envía JSON válido.
 - Petición GET a ese endpoint => Recibió {"error": "método no permitido"} con 405, correcto según código.
- Se hicieron pruebas con la API de login:
 - Usuario correcto => success True.
 - Contraseña incorrecta => success False.
 - Faltando campo => dio KeyError en código (no manejado), por lo que se añadió .get para evitar KeyError. Tras ajuste, con JSON incompleto devuelve success False al no encontrar user o password.
- En general, las funciones del backend pasaron estas pruebas simples, indicando que la lógica era sólida para datos esperados.

- **Prueba de interfaz web en aislamiento:**

- Se ejecutó la aplicación Ionic en modo dev con un servidor JSON simulado (mock). Esto para verificar la UI sin depender del backend. Se simulaban datos de herramientas y se comprobó que la lista mostrara ítems, que los botones de menú navegaran, etc.
- Resultado: La UI cargó correctamente en un navegador móvil y PC. Los layouts se adaptaron (la lista se veía como tarjetas en móvil y tabla en desktop, ajustado con CSS).



- Se identificaron algunas traducciones pendientes (los textos estaban en español en su mayoría, acorde al proyecto; se revisó ortografía).
- Esta prueba permitió pulir la apariencia antes de conectarla al backend real.

9.2 Pruebas de sistema integrales:

Una vez verificados los módulos, se procedió a probar el **sistema completo** en escenarios que emulan el uso real:

- **Escenario 1: Retiro simple de una herramienta por un usuario.**

- *Procedimiento:* Un miembro del equipo (con pulsera RFID) pasa por el portal tomando una herramienta con tag.
- *Expectativa:* El sistema debe registrar quién y qué herramienta salen, y reflejarlo en la interfaz.
- *Observaciones:* Al pasar, se escuchó un pitido (el lector tenía beep para lecturas). El ESP32 mostró 2 tags detectados (usuario y herramienta). En la BD aparecieron 2 registros con timestamps idénticos. En la web (admin), se interpretó visualmente que correspondían.
- *En la app de gestión,* aunque no estaba automatizada la asociación, se pudo ver en la sección de registros los códigos en esa hora. Con la tabla de referencia a mano, se confirmó “Juan sacó Taladro a las 10:05”.
- *Resultado:* Éxito. Puntos de mejora: vincular esos en uno solo evento para facilitar lectura, pero funcionalmente se cumplió: la información quedó almacenada.

- **Escenario 2: Devolución de la herramienta por el mismo usuario.**

- *Procedimiento:* El mismo usuario regresa con la herramienta y pasa por el portal.
- *Expectativa:* Registrar que la herramienta volvió.
- *Observaciones:* Nuevamente se registraron 2 lecturas (usuario, herramienta) con hora posterior.
- *Verificación:* Al consultar los registros, se tiene historial consistente: se ve la secuencia del tag de la herramienta a salida y entrada. A falta de una capa de lógica de “prestamos”, manualmente se interpretó que desde la salida hasta la



entrada, esa herramienta estuvo fuera.

- *Resultado:* Éxito.

- **Escenario 3: Múltiples herramientas a la vez.**

- *Procedimiento:* Un usuario toma 3 herramientas juntas y sale.
- *Expectativa:* Deben leerse las 3 etiquetas de herramientas y la del usuario en un solo paso.
- *Observaciones:* En la primera prueba, se detectaron 2 de 3 herramientas (posiblemente la tercera estaba apilada de tal forma que su tag quedó tapado por metal). Esto permitió recalibrar la posición de la etiqueta extraviada y repetir. En segunda pasada, se lograron las 3 lecturas.
- *El ESP32 envió casi simultáneamente 4 POST (1 usuario + 3 herramientas).*
- El backend registró las 4 entradas (vimos en logs 4 peticiones casi en el mismo segundo). Todos guardados.
- *En la interfaz,* se apreció que para una misma marca de hora aparecieron 4 códigos. Efectivamente correspondían a UsuarioX, HerrA, HerrB, HerrC.
- *Resultado:* Parcialmente exitoso. Detectamos la importancia de la posición de herramientas para multi-lectura. En uso real, se podría recomendar pasar las herramientas separadas en caso de que sean muchas, o implementar varias antenas para cubrir ángulos muertos. Pero el sistema demostró poder manejar múltiples lecturas.

- **Escenario 4: Usuario no autorizado o etiqueta desconocida.**

- *Procedimiento:* Pasar por el portal una tarjeta RFID que **no** esté registrada (o una herramienta sin tag RFID).
- *Expectativa:* El sistema igual registra el tag, ya que en esta versión no tiene lista de bloqueo. Luego se notará que es desconocido.
- *Observaciones:* Se usó una tarjeta UHF distinta (cuyo ID no teníamos mapeado). El backend guardó ese tag. En la interfaz, solo aparece un código más.
- *Interpretación:* Como no sabíamos qué era, se dedujo que es algo no inventariado. En un desarrollo futuro, se podría marcar como “tag no



reconocido”.

- *Resultado:* El sistema técnicamente funcionó (registró todo), pero resalta la necesidad de una base de datos de herramientas autorizadas para poder avisar al operador: “Tag XYZ detectado no corresponde a ninguna herramienta registrada”.
- Esto se anotó como mejora: agregar validación en recibir_tag para si el tag no está en lista permitir igual registro pero quizá con flag, o descartarlo. Depende de política (podría ser un objeto no autorizado que alguien intenta sacar).

- **Escenario 5: Caída de red durante operación.**

- *Procedimiento:* Desconectar el WiFi a mitad de un tránsito por el portal.
- *Expectativa:* El ESP32 no podrá enviar en el momento, pero no debe colgarse; idealmente reintente o almacene y envíe luego (no implementado del todo).
- *Observaciones:* Se hizo pasar un tag y simultáneamente se apagó el router. El ESP32 al intentar urequests.post tiró excepción. La capturó e imprimió error. Ese registro obviamente no llegó al servidor. Cuando se restableció WiFi (router on), el ESP32 se reconectó (reiniciado manualmente, porque micropython no reintenta solo sin código). Esa lectura se perdió del sistema.
- *Resultado:* Aquí se comprobó la importancia de robustez:
 - Solución simple: en MicroPython, implementar cola local. Por ejemplo, si falla envío, guardar el JSON en una lista e intentar más tarde. Dado el tiempo, no se implementó en proto, pero se sugiere.
 - Afortunadamente, en entorno real, la red local es controlada y es poco probable que esté caída. Si la Pi estuviese apagada, sería análogo. Por eso hay que asegurar la Pi siempre encendida cuando pañol abierto.
- *Conclusión:* El sistema es vulnerable a pérdida de datos en caso de caída de red/servidor en el instante justo de paso. Mitigación: añadir buffer en ESP32.

- **Escenario 6: Carga prolongada y estabilidad:**

- *Procedimiento:* Dejar el sistema funcionando varias horas y realizar muchos pases.
- *Expectativa:* Que no haya leaks, que SQLite maneje crecientes registros, etc.



- *Observaciones:* Se simularon ~100 eventos (50 retiros y 50 devoluciones) utilizando un script repetitivo de tags (sin personas reales, se pasaba repetidamente unas 5 tarjetas y 10 tags herramienta).
- El ESP32 manejó bien, sin ralentizar. La Raspberry Pi con Django en debug mostró incremento en el tamaño de db.sqlite3 pero insignificante (apenas unos KB por registro, total unos cientos de KB).
- Consultar la base de datos con `SELECT count(*)` mostró el número correcto (coincidente con envíos).
- Se *monitoreó* el uso de CPU/RAM en Pi: CPU ~3% con server idle, subió a 5-10% durante ráfagas de 4-5 posts por segundo (nada preocupante). RAM < 100MB para Python. Esto indica que en escalas del proyecto (decenas a cientos de eventos diarios) no hay problema.
- *Al final de la prueba prolongada*, el sistema seguía responsivo. No se observaron cuellos de botella.

9.3 Resultados esperados en entorno real:

Dado lo observado en pruebas, podemos establecer los siguientes resultados esperables al desplegar SICAP en un pañol real:

- **Detección exitosa de entradas/salidas:** Cada vez que un operario pase con una o más herramientas, el sistema registrará prácticamente en tiempo real la ocurrencia. Se espera >95% de tasa de lectura correcta de tags si los tags están bien colocados y no hay interferencias severas. En casos de no lectura (por ejemplo, etiqueta dañada o obstruida), el procedimiento de la institución podría indicar pasar de nuevo la herramienta individualmente para asegurar captura.
- **Actualización inmediata del inventario:** Apenas registrada una salida, la herramienta figurará como “no disponible” en el inventario (suponiendo que la app gestione ese estado). Esto permite a cualquier otro usuario saber que la herramienta está en uso.
- **Trazabilidad completa:** Quedará un registro permanente de quién retiró qué herramienta y a qué hora, así como cuándo fue devuelta. Esto facilita auditorías: por ejemplo, si falta una herramienta, se puede revisar en la base quién fue el último en tenerla .
- **Ahorro de tiempo y reducción de errores:** Al eliminar la necesidad de anotar manualmente en planillas cada préstamo/devolución, se espera un ahorro significativo de tiempo para el encargado del pañol, y se evitan errores humanos como olvidarse de anotar o escribir datos incorrectos . Las pruebas demostraron que el sistema funciona



de forma automática sin intervención; el personal solo necesita monitorear eventualmente la aplicación web.

- **Seguridad mejorada:** Es menos probable que alguien retire una herramienta sin permiso sin quedar registrado. Si bien SICAP no impide físicamente el movimiento de herramientas, la sola presencia del control puede disuadir extracciones indebidas. Además, al ser visible quién tiene cada cosa, hay mayor responsabilidad de los usuarios (se espera fomentar la devolución oportuna para no quedar registrados negativamente).
- **Facilidad de uso:** Operar SICAP no requiere pasos adicionales para los operarios más que llevar sus herramientas pasando por la puerta. En las pruebas, los usuarios no tuvieron que detenerse ni hacer nada especial; el sistema es transparente a su flujo normal de trabajo. Para los administradores, la interfaz web planeada será intuitiva, mostrando información relevante sin requerir conocimientos técnicos.

Reacciones y validación final: El sistema fue presentado a instructores y compañeros. Se hicieron demostraciones en vivo simulando un día de trabajo: distintos estudiantes tomaban herramientas al inicio de clase y las devolvían al final, mientras en una pantalla se proyectaba el panel de control mostrando los movimientos en tiempo real. La demostración fue exitosa, validando el cumplimiento de los objetivos:

- Se pudo demostrar que ninguna herramienta salía del pañol sin quedar asentada la salida.
- Cuando se quiso cerrar el pañol, se consultó la lista de herramientas fuera y se supo exactamente quién debía devolver qué (aunque esta asociación se hacía manual en esta versión, conceptualmente se entendió).
- Un posible despiste (un estudiante intentó llevarse una herramienta a casa sin permiso como prueba) fue evidenciado por el sistema, ya que quedó registrado que esa herramienta no había regresado (falta implementar alarma, pero con revisión manual se detectó). Esto confirmó la **utilidad real**: antes podría haberse escapado a la atención.

En conclusión, las pruebas certifican que SICAP es un **prototipo funcional** que logra sus objetivos de control automatizado. Las mediciones de desempeño y los casos de uso cubiertos indican que el sistema puede manejar el volumen y frecuencia típicos de un pañol escolar (decenas de movimientos diarios) con amplia capacidad de sobra. Se han identificado algunas áreas de mejora (principalmente en software: robustez de reconexión, asociación de registros y usabilidad de la interfaz) que se discuten en la sección final, pero ninguna de ellas impidió que el sistema cumpliera con su propósito en el contexto actual.



9. Conclusión, Mejoras Propuestas y Proyección Futura

Conclusión general: El proyecto SICAP logró integrar exitosamente diversas tecnologías en un sistema único que **automatiza el control de activos de un pañol** de manera eficiente e inteligente. A lo largo de esta carpeta técnica se detalló su concepción, diseño e implementación, demostrando que con recursos relativamente accesibles (un ESP32, un lector RFID, una Raspberry Pi) es posible replicar funcionalidades propias de sistemas industriales de gestión de inventario. Se cumplieron los objetivos planteados: se eliminó la carga manual de registros, se obtuvo trazabilidad en tiempo real de las herramientas, y se proporcionó una interfaz centralizada para monitorear la información. Además, desde el punto de vista pedagógico, el proyecto brindó a sus desarrolladores una experiencia integral en ingeniería de sistemas, fortaleciendo habilidades técnicas y de trabajo en equipo.

Beneficios alcanzados:

- **Reducción de pérdidas y extravíos:** Al saber siempre qué herramientas están fuera y con quién, se minimiza la posibilidad de que queden olvidadas o se pierdan sin darse cuenta. Se espera que con SICAP implementado, las pérdidas de equipamiento se reduzcan drásticamente (idealmente a cero).
- **Ahorro de tiempo y mejora de procesos:** El encargado del pañol ya no debe mantener planillas ni perseguir a los usuarios para que firmen retiros; el proceso es automático. Esto libera tiempo para otras tareas y agiliza la entrega/devolución.
- **Datos para toma de decisiones:** Con la base de datos de registros, se pueden analizar patrones de uso: qué herramientas son más utilizadas (y quizá requieran duplicados), en qué horarios hay más demanda, quién necesita más capacitación por extraviar cosas, etc. La dirección de la institución puede basar compras futuras o políticas de uso en estos datos.
- **Escalabilidad modular:** El sistema fue diseñado modularmente, lo que permite ampliarlo. Por ejemplo, se podría añadir otro portal RFID en otro taller y conectarlo al mismo servidor (solo implicaría un nuevo ESP32 enviando a la API; la base de datos podría registrar de qué sector proviene con un campo adicional).
- **Replicabilidad:** Gracias a la documentación extensa (esta carpeta) y el uso de componentes open-source, SICAP es replicable en otras instituciones con necesidades similares. El hecho de usar tecnologías estándar (Django, Angular, etc.) facilita que otros desarrolladores entiendan y modifiquen el sistema a su contexto.

Mejoras propuestas (trabajos futuros):

A pesar del éxito, al ser un prototipo existen varias mejoras y características adicionales identificadas que podrían implementarse para robustecer y ampliar SICAP:



1. **Asociación de tags con entidades (herramientas/usuarios):** Actualmente el sistema trabaja con códigos de tags “crudos”. Sería muy útil contar con un catálogo en la base de datos que mapeara cada tag de herramienta a su descripción (nombre, tipo, etc.) y cada tag de usuario a un nombre de persona. De esta forma, la interfaz web podría mostrar directamente mensajes comprensibles como “*Martillo (ID123) retirado por Juan Perez*” en lugar de códigos hexadecimales. Esto se lograría añadiendo modelos Herramienta y quizás utilizando el campo Pañol y usuarios Django para los operarios.

2. **Registro de eventos compuesto (préstamo/devolución):** Una mejora lógica sería que el backend interprete lecturas simultáneas de un usuario + herramienta(s) como un solo evento de “préstamo” o “devolución”. Esto podría hacerse añadiendo lógica de agrupamiento temporal: por ejemplo, si llegan múltiples registros con la misma marca de tiempo (o en ventanas de 1-2 segundos), combinarlos. Otra opción es cambiar la estrategia: en lugar de que el ESP32 envíe inmediatamente cada tag, podría acumular durante 1 segundo todo lo leído tras detectar el primer tag, y luego enviar un único payload JSON con, por ejemplo, {"user": "U12345", "tools": ["T99999", "T88888"]}. El backend podría entonces crear un registro compuesto. Esto haría más sencillo el seguimiento de préstamos individuales (y permitiría marcar la devolución asociada al préstamo original).

3. **Alertas y alarmas:** Una característica valiosa sería incorporar alarmas cuando ocurre algo fuera de lo normal. Por ejemplo:
 - *Herramienta no devuelta a tiempo:* Establecer un tiempo máximo de préstamo (p. ej. 1 día). Si pasado ese tiempo no hay registro de devolución, el sistema podría enviar un correo al encargado o generar una notificación en la app (“*La herramienta X retirada por Y no ha sido devuelta en el tiempo esperado*”).
 - *Intento de retiro no autorizado:* Si un tag de herramienta es detectado sin un tag de usuario acompañándolo (lo cual podría indicar que alguien sacó una herramienta burlando credenciales), se podría sonar una alarma audible en el pañol o enviar alerta. Similarmente, si se detecta un usuario retirando muchas herramientas a la vez cuando no debería, etc.
 - *Alarma anti-hurto en tiempo real:* Con hardware adicional (como una baliza o buzzer conectado a la Pi o al ESP32), se podría emitir una señal inmediata cuando se retira una herramienta fuera de horario permitido o por un usuario no autorizado.

4. **Seguridad informática:** Si bien en una red local cerrada la transmisión en claro de datos es aceptable, para endurecer seguridad:
 - Implementar **HTTPS** en el servidor (usando un certificado SSL local) y hacer que el ESP32 use urequests con URL https://. Esto complica un poco (MicroPython



tendría que tener soporte SSL y certificado, etc.), pero cifra los datos en tránsito, previniendo sniffing incluso interno.

- **Autenticación de la API:** Actualmente cualquiera en la red que conozca la URL podría enviar POSTs falsos. De cara al futuro, se debería proteger el endpoint. Opciones: una API Key secreta que el ESP32 incluya en los POST (y el servidor verifique), o un token JWT que se negocie una vez. O incluso verificar dirección MAC, aunque es en capa distinta. Un método sencillo: añadir un campo “clave”: “XYZ” en JSON y el servidor solo acepta si coincide con una clave compartida. Dado que la red es cerrada, no es crítico, pero es buena práctica.
 - **Roles de usuario en app:** Distinguir entre administradores (pueden ver todo, registrar nuevas herramientas, usuarios) y usuarios estándar (quizás un operario pueda entrar solo para ver qué tiene asignado, etc.). Django facilita esto con su sistema auth; habría que reflejarlo en Angular (quizás diferentes vistas).
5. **Extensión a múltiples ubicaciones:** Si la escuela tiene varios pañoles (electrónica, mecánica, etc.), se puede extender el modelo para cubrir todos con un solo sistema:
- Cada registro RFID indicaría qué pañol es (esto se podría identificar si se usan tags de zona distintos o, más fácil, desplegando múltiples ESP32 con endpoints diferentes, e.g. /api/v1/registro/panol1/, /panol2/, o incluir un campo “pañol”: “1” en JSON).
 - La base de datos Pañol que ya existe puede cobrar protagonismo, para separar registros y accesos por área.
 - La interfaz permitiría filtrar por pañol o tener paneles separados. Esto escalonaría SICAP a un sistema institucional de control de activos más amplio.
6. **Registro de mantenimiento y estado:** Se puede agregar funcionalidad para marcar herramientas dañadas o en mantenimiento. Por ejemplo, si una herramienta se rompe, el encargado podría desde la web app marcarla “En reparación” y aunque físicamente esté en el pañol, que el sistema la muestre como no disponible para préstamo.
7. **Integración con base de datos externa o ERP:** En entornos industriales, podría querer integrarse SICAP con sistemas de inventario más grandes. Django podría conectarse a una DB central o exponer servicios web. Por ahora no es necesario, pero saber que la arquitectura lo permite es positivo.
8. **Interfaz móvil para operarios:** Actualmente, la interfaz web está pensada para administradores. Sería interesante desarrollar una **app móvil para operarios** (quizás usando el mismo Ionic adaptado) donde cada usuario pueda iniciar sesión y ver qué herramientas tiene en préstamo, recibir notificaciones (“Debes devolver X antes de tal



día”), e incluso hacer solicitudes de herramientas. Esto ya trasciende el alcance original, pero es una proyección realista en ámbitos profesionales (gestión de stock bajo demanda).

9. **Hardware: Mejora de antenas y cobertura:** Si se quisiera cubrir un área más amplia o asegurar lectura desde cualquier orientación, se pueden añadir más antenas al portal (muchos lectores permiten 4 puertos de antena). También se puede aumentar potencia de emisión dentro de límites legales. Otra mejora de hardware: incorporar sensores de movimiento o puertas para detectar ingreso y así activar la lectura RFID solo en ese momento (para ahorrar energía y reducir lecturas fantasmas).
10. **Modo offline (cache local):** Actualmente, si el servidor está apagado, el ESP32 no guarda nada. Se podría implementar un backup offline: por ejemplo, que el ESP32 escriba en su flash interna los eventos mientras no hay conexión, y cuando recupera conexión los envía. Así se hace tolerante a fallos de red temporales. MicroPython tiene librería uos para escribir a un archivo; se podría usar.
11. **Uso de cámara/visión:** Un futuro ambicioso: complementar RFID con visión artificial. Ej: una cámara en la puerta que capture imagen de quien sale y verifique visualmente las herramientas (como redundancia o para casos en que falle RFID). Dado que ya hay un servidor Pi, se podría entrenar un modelo ligero para reconocer formas de herramientas... es complejo pero posible. Aunque tal mejora quizás excede necesidades prácticas, es una línea de investigación (algunos sistemas comerciales lo combinan).
12. **Documentación y mantenimiento:** Institucionalizar el uso de SICAP requerirá también crear manuales de usuario (para el encargado y para los operarios) y plan de mantenimiento: chequear etiquetas periódicamente, recalibrar antenas si es necesario, etc. Parte de la proyección es asegurarse que el sistema se pueda mantener con el recambio de personal: esta carpeta técnica servirá para que futuros encargados entiendan cómo funciona todo.

Proyección a futuro:

SICAP tiene potencial para transformarse en un producto más allá del proyecto escolar. Por ejemplo:

- Podría implementarse en **escuelas técnicas de toda la provincia** para gestionar sus talleres. Adaptando algunos parámetros, sería casi plug-and-play para ellos.
- En **pequeñas empresas** con almacén de herramientas, SICAP les brindaría control sin invertir en sistemas caros.



- Incluso en **bibliotecas públicas** o universitarias pequeñas, se podría aplicar (cambiando quizás a RFID HF para libros, pero el software es adaptable; de hecho, la sección “¿Dónde se puede aplicar?” ya lo contempla).
- Como proyecto open-source, SICAP podría subirse a una plataforma tipo GitHub para que la comunidad contribuya a mejoras (nuevas features, soporte a distintos hardware, etc.). El espíritu modular de Django/Angular favorece esta colaboración.

En el corto plazo, la prioridad sería implementar las mejoras de software mencionadas (sobre todo la asociación de nombres a tags y la generación de eventos préstamo/devolución, ya que eso hará la solución mucho más usable para personal no técnico). También, pulir la interfaz web para que en la presentación final esté completamente funcional y amigable.

Reflexión final: El desarrollo de SICAP ha permitido comprobar la viabilidad de introducir **tecnología RFID e IoT en entornos educativos** para resolver problemas concretos. Se ha pasado de una situación de desorden y falta de responsabilidad en el pañol, a un sistema inteligente que promueve la organización y la responsabilidad compartida sobre los recursos . Más allá de la tecnología, SICAP representa un cambio cultural: inculcar en los usuarios la importancia de devolver lo que se toma y cuidar las herramientas, sabiendo que hay un control pero también un apoyo (porque si olvidan algo, el sistema se los recordará).

Para los desarrolladores, fue un viaje de aprendizaje intensivo, aplicando conocimientos de electrónica, programación, bases de datos y redes adquiridos a lo largo de la carrera. El resultado obtenido demuestra que con creatividad y dedicación, es posible abordar desafíos logísticos con soluciones innovadoras de bajo costo. Este proyecto sienta las bases para futuros desarrollos aún más ambiciosos dentro del campo de la automatización y control, y deja abierta la invitación a mejorarlo y adaptarlo a nuevos horizontes.

En síntesis, SICAP cumple con su nombre: es un **Sistema Integral** que unifica hardware y software para el **Control Automatizado** de activos, y su implementación exitosa en el pañol escolar augura **un futuro promisorio** donde estas ideas se expandan y evolucionen, beneficiando tanto a instituciones educativas como a cualquier organización que requiera orden y eficacia en la gestión de sus recursos físicos.

